



Testbed 2026 Digital Twin as a Service (Phase 2)

Research topic #4: App-Store / Recipes

Final Report

Future Insight

Future Insight Group BV - Grote Voort 207-G – 8041 BK Zwolle
www.futureinsight.nl – info@futureinsight.nl -- KVK 63664836

1. Summary 3
2. Introduction and scope 3
3. Use cases and bundled recipes 5
4. Architecture 8
4.1 One cook run, step by step 11
4.2 Design principle: one reader of the recipe, one plan for everyone 12
5. The recipe format: a CWL profile 15
5.1 Standard CWL we use 15
5.2 Extensions we needed 19
5.3 Data inputs vs parameters 21
5.4 Parameters: CWL types vs JSON Schema 22
5.5 Example 24
6. The Cookbook Registry 25
6.1 What the DAG contains 27
6.2 Design choices 30
7. The Cook Orchestrator 31
7.1 What happens when you click 'Execute' 31
7.2 Sync and async 35
7.3 Where a step runs 36
7.4 Where a failure came from 36
8. Catalogue and recipe metadata 37
8.1 Records Wrapper over CKAN 37
8.2 What a record contains 38
8.3 Recipe metadata in DCAT-AP-NL: what is missing, and what to add 39
9. The client: App Store UI 44
9.1 How the UI uses the recipe 45
9.2 Catalogue, recipe pages and home page 47
9.3 The Recipe Wizard 48
9.4 Status tab and saved results 51
9.5 Orchestrator editor 53
10. Plugfest and interoperability 54
11. Findings 58
Recipe format (CWL) 58
OGC API Processes 59
Catalogue and metadata 60
App Store operation 60
Access, delegation and cost 60
Questions raised in the stand-ups 61
12. Recommendations and future work 61
13. Collaboration 62
14. Availability, licensing and running the stack 63
Appendix A: Design decisions and turning points 66
Appendix B: Using the stack without the UI (API reference) 73

B.1 Endpoints....73
B.2 Our own apps....77
B.3 Examples....77
Appendix C: The playground recipes, annotated....81
C.1 playground-1: roofer building reconstruction....81
C.2 playground-2: Geolabs ZOO-Project sampler....85
C.3 playground-3: Varidian's Subsidence Analysis....89

1. Summary

We built a working, end-to-end App Store for recipes: a catalogue (OGC API Records over CKAN), a Cookbook Registry, a Cook that is itself an OGC API Process, and a web client. Recipes are written in CWL v1.2 and executed against our own processes and external testbed processes (roofer, Geolabs ZOO-Project, Varidian).

Our main conclusion: **CWL works as a recipe format, but not unextended.** CWL describes ingredients, order and wiring well. It cannot say where a step's OGC API Process lives, which alternative processes a hub may use, or describe process inputs as precisely as JSON Schema does. We added three small extension fields in a declared `nldt:` namespace to close those gaps; with them, all five bundled recipes pass the reference validator, `cwltool --validate`.

Other key findings:

- CWL's abstract `Operation` step maps one-to-one onto a remote OGC API Process.
- CWL's own `File` + `format` cleanly separates data ingredients from parameters, and maps onto OGC by-reference (`href`) inputs.
- OGC API Processes descriptions do not say machine-readably which inputs are datasets, so the recipe must declare it.
- Asynchronous execution with job polling is mandatory in practice for long-running processes.
- DCAT-AP-NL has no recipe concept yet; proposing the extension is the main open research item.
- Writing CWL by hand is hard for non-specialists; a guided editor is essential.
- Every part of the stack works from one plan: the Cookbook Registry turns a recipe into a DAG, and the editor, catalogue, Wizard, Status tab and Cook all ask it for that plan instead of reading the CWL themselves.
- An OGC API Process description already carries most of what a user needs to fill in a parameter (title, description, default, limits); the Wizard reads it when it shows the form, so recipes need not copy it.

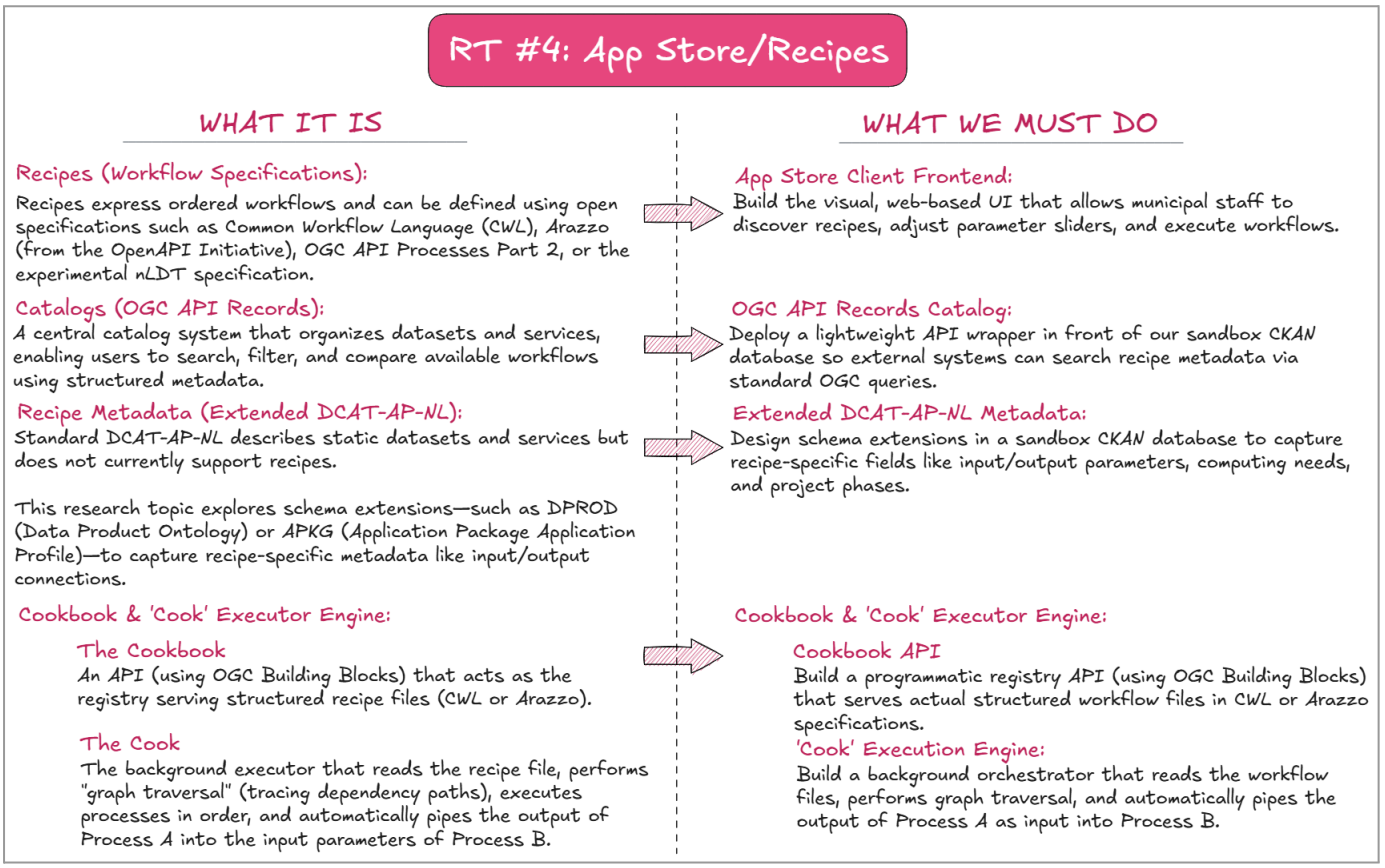
Not finished: DCAT-AP-NL recipe metadata is not implemented in the catalogue (section 8.3 sets out what a profile needs, based on what the testbed showed). Our OGC API Processes services were tested with OGC's official test suite, fixed where it found gaps, and re-tested on the live stack (section 10).

Live client: fi-group.github.io/nldt-tb2

2. Introduction and scope

This report covers Future Insight's work on Research Topic #4, App-Store / Recipes, of Geonovum's Testbed 2026 phase 2. The tender defines a recipe as a combination of data

sources (ingredients) and an ordered list of processes (cooking instructions) that solves a



real-world problem.

The table maps each component the tender (§7.3) asks for to what we built.

Tender task (§7.3)	**What we built**	**Standard**
Recipe	CWL `Workflow`; each step is an OGC API Process	CWL v1.2 + 3 `nldt:` extensions
Recipe metadata in extended DCAT-AP-NL	Gap analysis and proposal (section 8.3); not implemented	DCAT-AP-NL (open)
Catalogue	Records Wrapper: OGC API Records façade over CKAN	OGC API Records Part 1: Core

Tender task (§7.3)	What we built	Standard
Cookbook	Cookbook Registry: stores CWL, derives input schema and DAG	Purpose-built REST
Cook	Cook Orchestrator: itself an OGC API Process `cook`	OGC API Processes Part 1: Core 1.0
Data ingredients	Hub Datasets: the hub's datasets (live PDOK data, samples, saved results)	Plain REST
Client	App Store UI: catalogue, Recipe Wizard, Status, CWL editor	—

The UI and its error messages call the Cookbook Registry "Cookbook API Registry"; both names mean the same service.

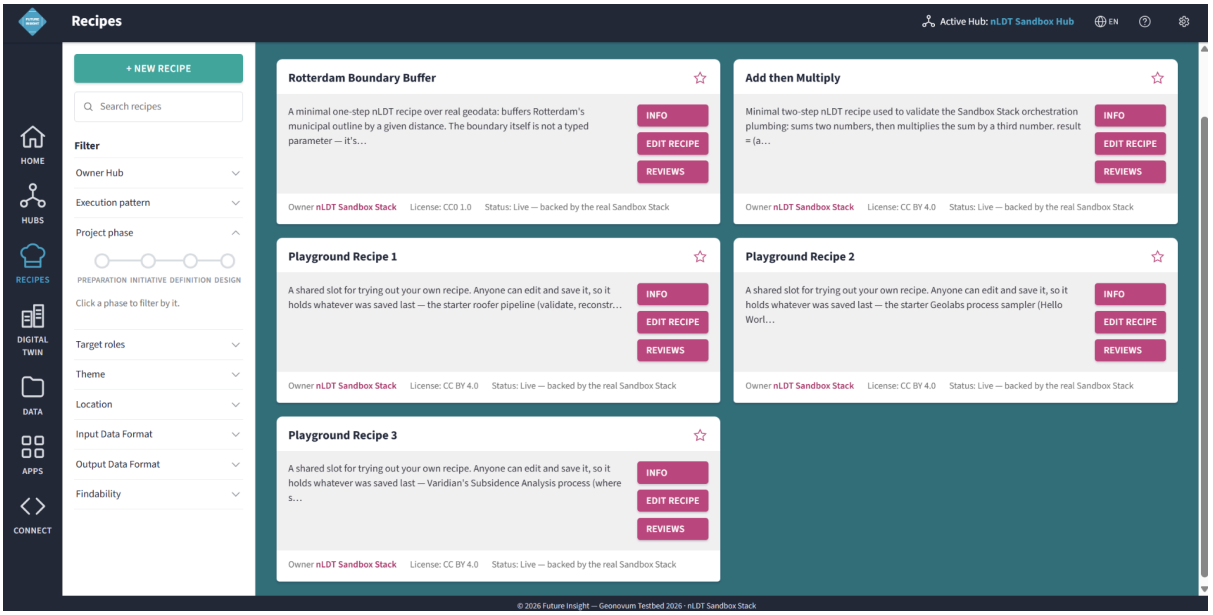
Out of scope: authentication (of users, and of the apps a recipe calls), pricing, ownership and versioning of recipes, and production hardening. The stack is a sandbox (section 14). Section 11 explains why authentication and pricing matter most for real use (findings 23–25).

Standards version. The tender encourages OGC API Processes v2.0. We implement Part 1: Core 1.0 on the wire, because its conformance classes are stable. Chaining is done through CWL inside the Cook, not through Part 3 (Workflows) nested execution.

3. Use cases and bundled recipes

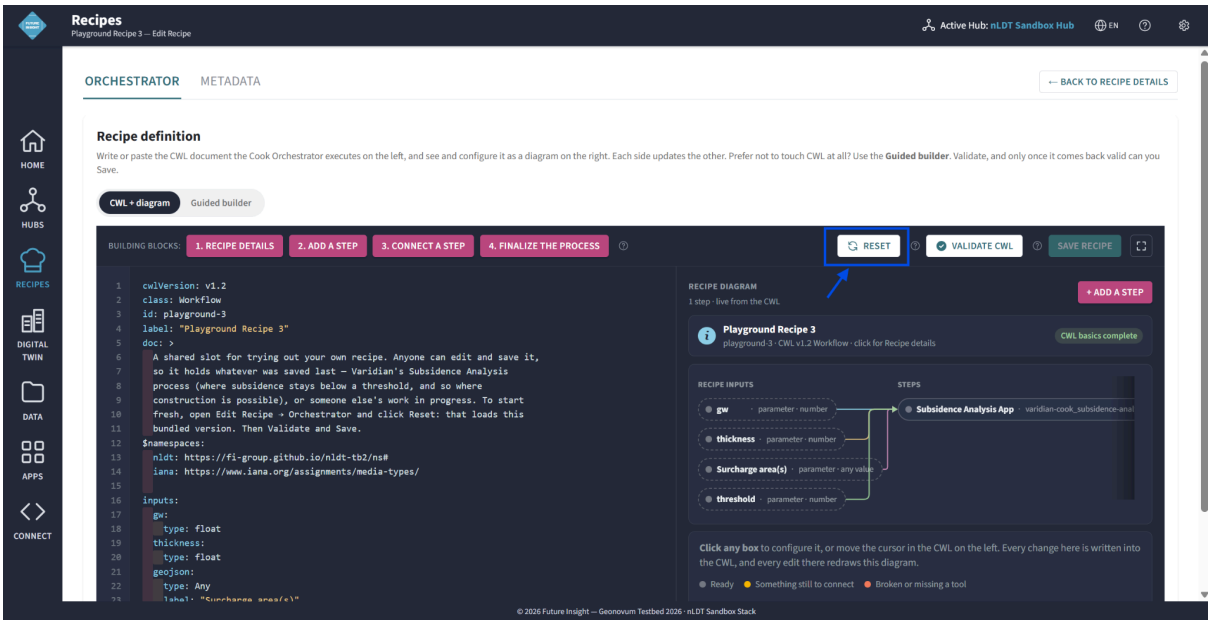
We proved the orchestration on a trivial recipe first, then moved to real geodata and external processes. The first recipe, `(a + b) × c`, is the smallest one that exercises the whole chain: discovery, retrieval, parameters, two steps, output piped to input, result.

Recipe	What it shows	Apps it calls
`recipe-rotterdam-buffer-demo`	Real geodata: buffers Rotterdam's municipal outline, read live from PDOK via Hub Datasets (one data input, one parameter)	Our own buffer tool
`recipe-add-multiply-demo`	The orchestration plumbing, `(a + b) × c`. Its `nldt:compatibleTools` allows swapping multiply for `multiply-three-numbers`, which adds a third input automatically	Our own add and multiply tools
`playground-1`	Roofer pipeline: validate point clouds → reconstruct buildings → convert format	Roofer's processes (staging server)
`playground-2`	ZOO-Project sampler: five processes (HelloPy, hellojs, Centroid, Buffer, GetArea), mostly in parallel, with one chain Buffer → GetArea	Geolabs' ZOO-Project processes
`playground-3`	Subsidence analysis: where subsidence stays below a threshold, and so where construction is possible (see below)	Varidian's OGC API Process



Playground slots: the state at the time of writing. The three playground slots were seeded for the plugfests, so participants could test their own processes in a recipe without our help. Anyone can open a playground recipe in the editor, change it and save it: there is no login, no ownership and no version history (*finding 21*). Saving overwrites the slot for everyone. The table above therefore describes the playground recipes *as they were when this report was written*. A reader who opens the live App Store later may find something else in these slots, or an earlier version. Appendix C keeps all three as they were, in full and annotated.

Two things bring a playground slot back to a known state:



- **Reset**, in the recipe editor, loads the slot's *bundled* recipe (the version shipped with the code, `GET /recipes/{id}/original`) into the editor. It only replaces

the text in the editor; nothing changes for anyone else until that text is validated and saved. After the second plugfest we made the recipes in the table the bundled versions of all three slots, so Reset brings back exactly what is described here. (Until then, `playground-3` was bundled as a heavily commented add → multiply template, meant as a starting point for one's own recipe.)

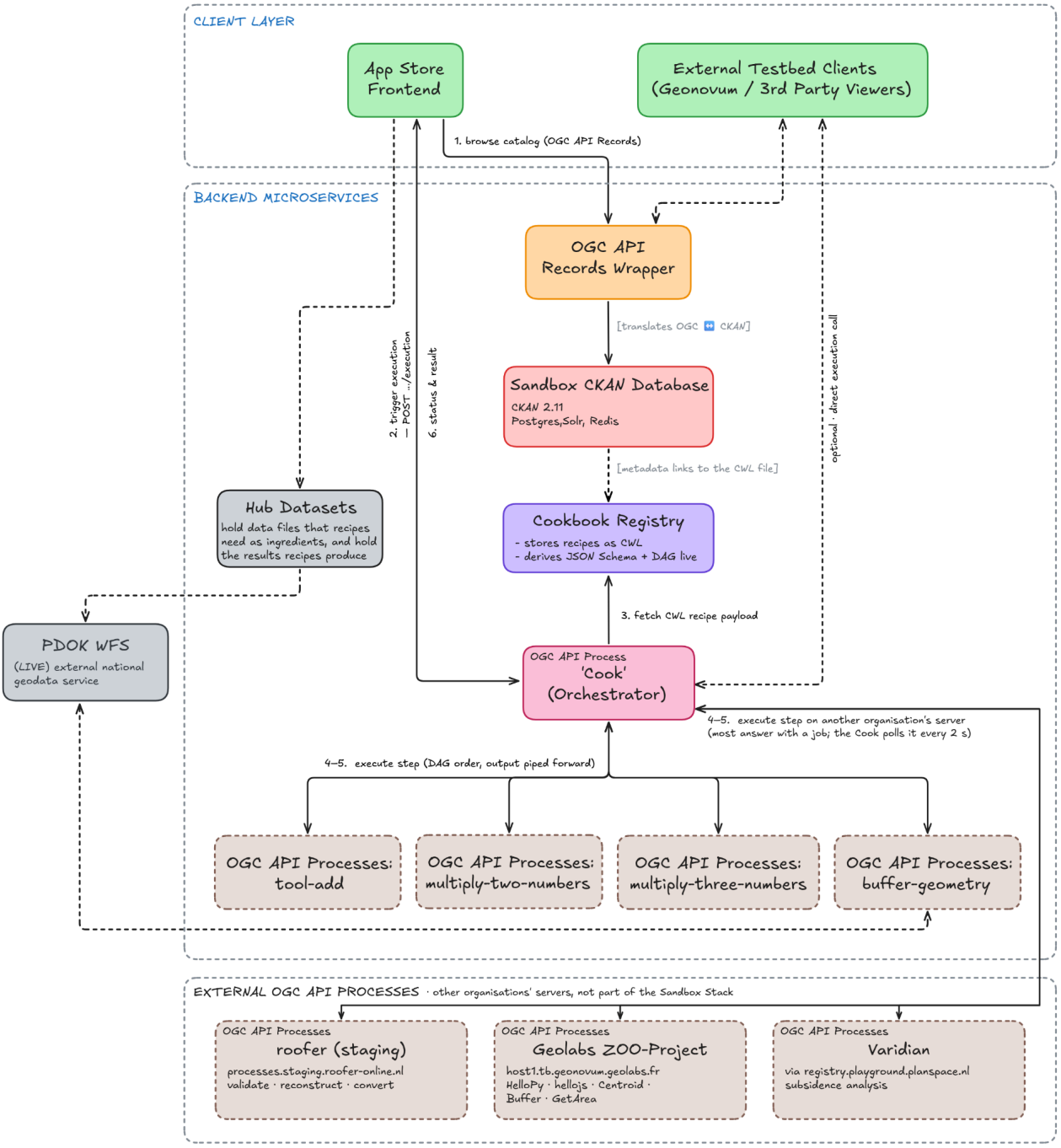
- **Re-seeding**, run when the stack is redeployed, stores every bundled recipe again, overwriting whatever was saved in the slots.

The two named recipes (`recipe-rotterdam-buffer-demo`, `recipe-add-multiply-demo`) can be edited in the same way; Reset and re-seeding restore them too.

Varidian's process in `playground-3`. At the second plugfest, Varidian put a one-step recipe around their own OGC API Process, `varidian-cook_subsidence-analysis`, into `playground-3`. The process is published at `registry.playground.planspace.nl`. It asks where subsidence stays below a threshold, using the Deltares Atlantis calculation module, and returns the area(s) where construction is possible as a GeoJSON FeatureCollection. Its four inputs are a groundwater level, a surcharge thickness, a threshold and the surcharge area(s) as GeoJSON. The recipe keeps the GeoJSON as a parameter, typed or pasted into the form, rather than a data ingredient picked from Hub Datasets, so that testers can try their own polygons straight away. Runs are saved in Hub Datasets. Varidian's process is itself a "cook", which makes it an example of *finding 15*: a recipe published as a process, used as one step in another recipe.

4. Architecture

The stack is nine services of our own plus CKAN 2.11 (with its Postgres, Solr and Redis), each in its own Docker container, on one private network behind an nginx gateway. The gateway publishes everything under one public address, `https://nldt-tb2.clearly.app`, with each service under its own path. The App Store UI is also published on GitHub Pages, at `https://fi-group.github.io/nldt-tb2/`, and calls the services at that public address. The browser calls each service directly; there is no backend-for-frontend.


CLIENT LAYER
App Store Frontend
External Testbed Clients (Geonovum / 3rd Party Viewers)
1. browse catalog (OGC API Records)
BACKEND MICROSERVICES
OGC API Records Wrapper
[translates OGC ↔ CKAN]
Sandbox CKAN Database
CKAN 2.11
Postgres,Solr, Redis
[metadata links to the CWL file]
Cookbook Registry
- stores recipes as CWL
- derives JSON Schema + DAG live
3. fetch CWL recipe payload
Hub Datasets
hold data files that recipes need as ingredients, and hold the results recipes produce
PDOK WFS
(LIVE) external national geodata service
2. trigger execution — POST …/execution
6. status & result
optional · direct execution call
OGC API Process
'Cook'
(Orchestrator)
4–5. execute step on another organisation's server
(most answer with a job; the Cook polls it every 2 s)
4–5. execute step (DAG order, output piped forward)
OGC API Processes: tool-add
OGC API Processes: multiply-two-numbers
OGC API Processes: multiply-three-numbers
OGC API Processes: buffer-geometry
EXTERNAL OGC API PROCESSES · other organisations' servers, not part of the Sandbox Stack
OGC API Processes
roofer (staging)
processes.staging.roofer-online.nl
validate · reconstruct · convert
OGC API Processes
Geolabs ZOO-Project
host1.tb.geonovum.geolabs.fr
HelloPy · hellojs · Centroid · Buffer · GetArea
OGC API Processes
Varidian
via registry.playground.planspace.nl
subsidence analysis

Service	Public address (under https://nldt-tb2.clearly.app)	Role	Standard
ui	/ (and https://fi-group.github.io/nldt-tb2/)	App Store UI, static files only	—
records-wrapper	/api/records	Recipe catalogue; translates OGC API Records ↔ CKAN	OGC API Records Part 1: Core
cookbook-registry	/api/cookbook	Stores recipes (CWL), derives input schema and DAG, validates	Purpose-built REST
cook-orchestrator	/api/orchestrator	Executes a recipe: process cook, jobs, sync/async	OGC API Processes Part 1: Core 1.0
tool-add, tool-multiply, tool-multiply3	/api/tools/add, /api/tools/multiply, /api/tools/multiply3	Trivial numeric processes, one codebase, three configurations; sync only	OGC API Processes Part 1: Core
tool-geo-buffer	/api/tools/geo-buffer	Buffers GeoJSON by N metres (via EPSG:28992), by value or by	OGC API Processes Part 1: Core

Service	Public address (under https://nldt-tb2.clearly.app)	Role	Standard
		reference; sync only	
hub-datasets	/api/hub	The hub's ingredients and saved run results	Plain REST (deliberately no OGC claim)
ckan (+ Postgres, Solr, Redis)	Not public: reached only through the Records Wrapper	CKAN 2.11, the real store behind the wrapper	CKAN Action API
gateway	https://nldt-tb2.clearly.app	One origin: UI at /, services under /api/*	—

Every service and tool can be opened directly in a browser, and the whole solution can be tested without the UI (Appendix B lists the endpoints, with examples). For example, the Cook's process description is at: https://nldt-tb2.clearly.app/api/orchestrator/processes/cook,

and the buffer tool's at:

https://nldt-tb2.clearly.app/api/tools/geo-buffer/processes/buffer-geometry.

External processes and data: roofer (staging), Geolabs ZOO-Project, Varidian, and PDOK WFS (Kadaster).

4.1 One cook run, step by step

A run has five stages: find the recipe, bind it in the Wizard, start the cook job, execute the steps, and read the status and result.

1. The client lists recipes from the Records Wrapper (`GET /collections/recipes/items`). Each record links to its CWL in the Cookbook Registry.
2. The client asks the Registry for the recipe's input schema and DAG, lists matching Hub Datasets, and the user binds tools, data and parameters in the Recipe Wizard (section 9.3).
3. On the Status tab, Execute starts the `cook` process with `{recipeId, parameters, toolOverrides}`, asking for asynchronous execution, and gets a job back straight away.
4. The Cook fetches the DAG from the Registry and executes each step in dependency order, independent steps in parallel, piping outputs into the next steps' inputs (section 7.1).
5. The client polls `/jobs/{id}` every 2 seconds, showing each step's progress, then reads the result plus a per-step trace. The result is saved to Hub Datasets automatically.

4.2 Design principle: one reader of the recipe, one plan for everyone

A recipe is written once, as CWL, and then used in many places: the editor checks it, the catalogue shows it, the Wizard explains it, the Status tab summarises it and the Cook runs it. If each of those read the CWL itself, each would need its own CWL parser, and sooner or later two of them would disagree about what a recipe means.

So only the Cookbook Registry reads CWL. Everything else asks the Registry one question, "what is the plan for this recipe?", and gets back the same kind of answer: the recipe's **DAG** (directed acyclic graph) plus the JSON Schema of its inputs. Whether the reason for asking is to check a draft, explain the recipe to a person, draw a picture or run it, the answer is the same plan.

Why a DAG, and not the CWL itself? CWL is written for people. Steps appear in whatever order the author wrote them, and the file carries labels, descriptions, comments and extension fields. The machine that runs a recipe needs something else: which step must wait for which, and where each input of each step comes from. For example, a step written *first* in the file may need the output of a step written *second*. In CWL that dependency is only implied, by an input written as `x: add/sum` ("take output `sum` of step `add`"). In the DAG it is explicit: steps come in an order in which every step comes after the steps it depends on, and each input says whether it comes from a recipe input (`workflow_input`) or from another step's output (`step_output`). A cycle (two steps waiting for each other) cannot be put in such an order, so the Registry rejects it. Section 6.1 shows the DAG's fields and an example.

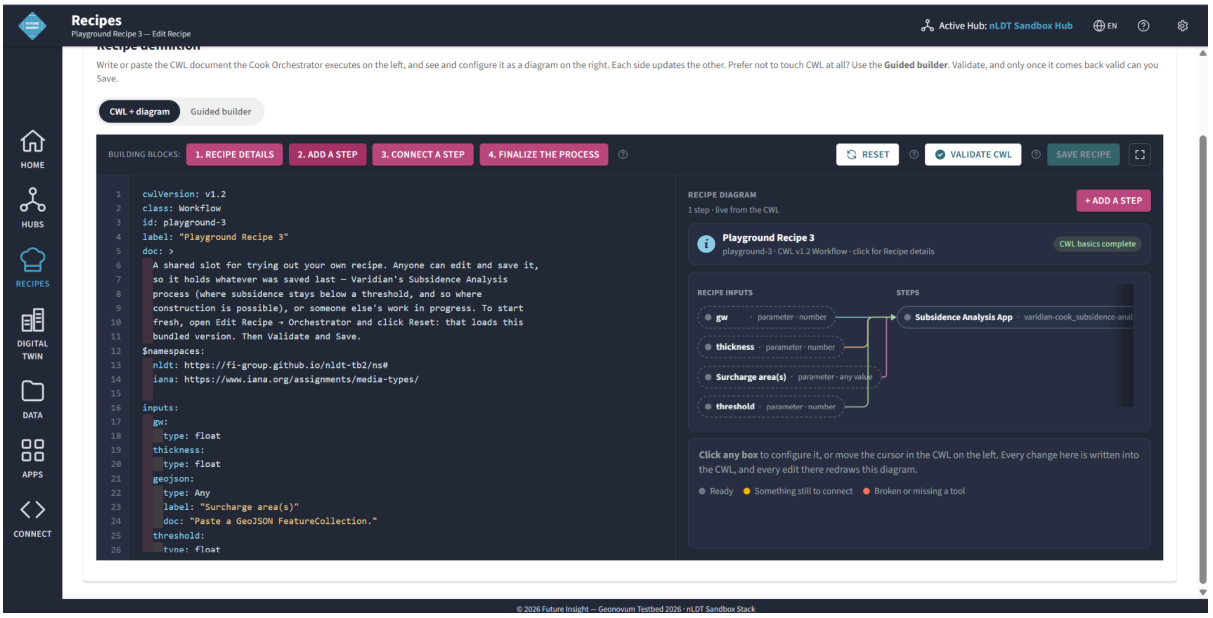
Who asks for the plan, and when. The Registry derives the DAG fresh on every request and never caches it, so every answer reflects the recipe as it is stored at that moment.

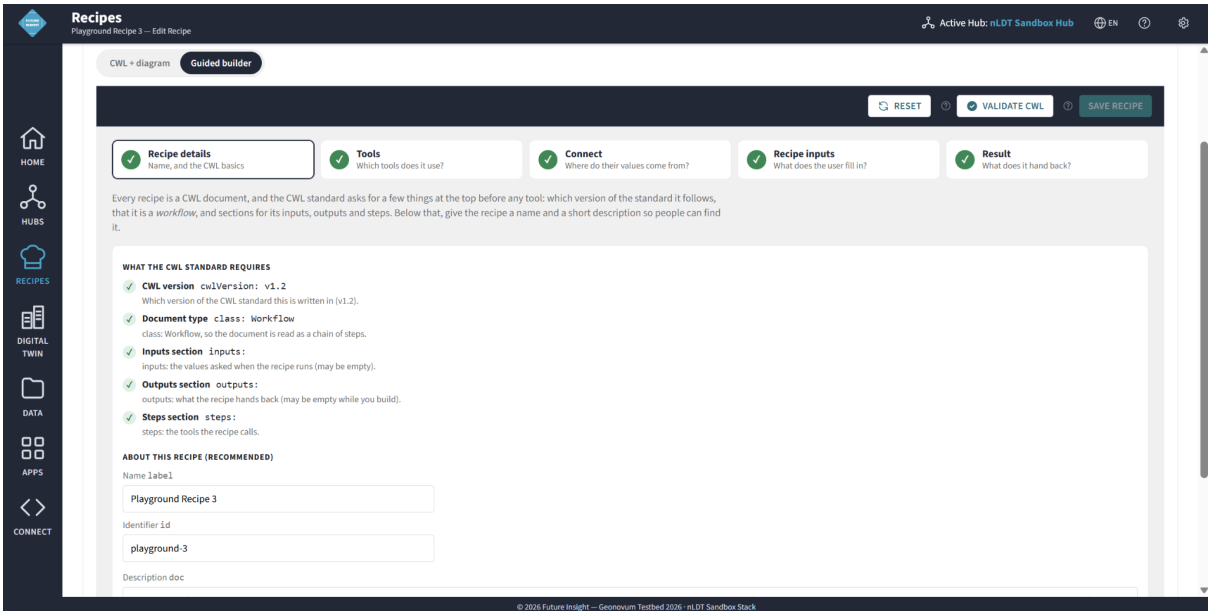
#	Who	When	Asks the Registry	Uses the answer to
1	A recipe author, in the editor	Clicks **Validate**	`POST /recipes/validate` (the draft text, not stored)	Show whether the draft is valid, its inputs and its step order; only then enable Save
2	The Registry itself	A recipe is **saved**	— (builds the DAG internally)	Refuse to store any recipe it cannot turn into a DAG
3	The catalogue and recipe pages	The recipe list loads	`GET /recipes/{id}/dag`, once per recipe	Know which inputs are data ingredients (`data_inputs`), for the recipe cards and the Wizard
4	The Recipe Details tabs	The page opens	`GET /recipes/{id}/dag`	Draw the recipe graph; list the apps the recipe uses on its Compatible Apps tab
5	A hub in the Recipe Wizard	Opening the Wizard, and after every app swap	`POST /recipes/{id}/effective` (with the hub's tool swaps)	Draw the graph on every step, list the steps and their apps, build the Data Ingredients and Parameters steps
6	A hub on the Status tab	Opening the tab, right before Execute	`POST /recipes/{id}/effective`	Draw the graph, show the values that will be sent, block Execute while

#	Who	When	Asks the Registry	Uses the answer to
				anything required is missing
7	The Cook	**Execute** is clicked	`GET /recipes/{id}/dag`, or `POST …/effective` when the hub swapped an app	Run the steps in order, and know where each input comes from

"Effective" means the recipe with a hub's per-run app swaps applied (section 5.2), computed for that request and never stored. The Wizard, the Status tab and the Cook ask for the same effective plan, so what a hub sees is exactly what the Cook runs.

One exception, by design. The editor's diagram and its guided builder read the CWL text in the browser, because their job is to edit that text line by line, including comments. They never decide what a recipe means: Validate and Save always go to the Registry.





5. The recipe format: a CWL profile

A recipe is a CWL v1.2 `Workflow` whose steps each run an inline `class: Operation`: CWL's own abstract, non-executable step. The Operation's `id` is the OGC API Process identifier. We chose CWL over Arazzo and a custom JSON format because it is a mature, externally governed standard for chaining discrete steps.

OGC's *Best Practice for Earth Observation Application Package* (OGC 20-089)[1] also packages CWL workflows for OGC API Processes, but differently. It uses `CommandLineTool` steps with container images that are *deployed* to a platform (Processes Part 2). Our recipes use `Operation` steps that *call* processes that already run elsewhere.

5.1 Standard CWL we use

CWL construct	Used for
`class: Workflow`, `cwlVersion: v1.2`	The recipe; `id`, `label`, `doc` seed the catalogue id, title and description on first load

[1] https://docs.ogc.org/bp/20-089r1.html

CWL construct	Used for
`inputs`	Parameters (`float`, `int`, `string`, `enum`, arrays, `Any`, optional, `default`) and data inputs (`File` / `File[]` + `format`)
`outputs` + `outputSource`	What the recipe returns
`steps` with `run: {class: Operation, id: …}`	One step = one OGC API Process
step `label` and the Operation's own `label`	Two different names: what the step does in this recipe, and the app that does it (see "Naming a step and its app" below)
step `in:` / `out:`	Wiring: short form (`x: add/sum`) or long form (`source:` + `valueFrom:`); output of process *a* becomes input of process *b*
`format` (IANA media types via `$namespaces`)	Media type of a data input; also used non-standardly on non-File process inputs and outputs (see below)
`valueFrom` (four patterns)	Reshaping a value for a specific process
`requirements`	`StepInputExpressionRequirement` and `InlineJavascriptRequirement`, declared where `valueFrom` is used

The Registry reads CWL with a purpose-built parser, not a general CWL engine. It treats unsupported features in two ways:

- **Rejected with an error:** a `cwlVersion` that is not `v1.x` (any `v1.x` is accepted, not only v1.2), non-Workflow documents, non-Operation steps (`CommandLineTool`, `ExpressionTool`), list-form `steps:` or `in:`, cycles, unknown step references, unsupported `valueFrom` expressions, and the retired `x-` extension names.
- **Silently ignored:** `scatter`, `when` and `$import`/`$include`. A recipe using `when` passes Validate, and the step then runs every time.

CWL expressions are JavaScript; we never execute them. Four `valueFrom` forms are recognised by pattern-matching: a field path, a `map` over a list picking a field, a `map` building an object, and an object template.

`requirements`: declared for CWL standard, ignored by our stack.

> CWL requires a recipe to declare the features it uses. A step input with `valueFrom` needs `StepInputExpressionRequirement`, and an expression that is real JavaScript, such as the `map` lines in the roofer recipe, also needs `InlineJavascriptRequirement`; a plain field path such as `$(self.model_3d_id)` does not.
>
> Neither the Registry nor the Cook reads these declarations: a recipe without them validates and runs the same in our stack. We declare them anyway, so that our recipes stay standard CWL: without them, `cwltool --validate` rejects a recipe that uses `valueFrom`. The Registry does not check for them either, another case of *finding 7*. Of the bundled recipes, only `playground-1` (roofer) needs them (Appendix C).

The Registry checks structure and wiring, not types: it checks that every connection points at something that exists, but not that the value sent fits what the other end expects. The reference validator, `cwltool --validate`, does check this, and it caught a real mistake the Registry had accepted during the first Plugfest. In the roofer recipe (`playground-1`), the recipe input `formats` was declared as a list of fixed choices (a CWL `enum`: `gpkg` or `cityjson`), but the convert step declared the process input it feeds as a list of free text (`string[]`). For CWL those are different types, so the connection between them is invalid, even though roofer itself would have accepted the values. We fixed the step's declaration to the same list of fixed choices.

Naming a step and its app. CWL gives every step two places for a name, and we use them for two different things:

Where in the CWL	Example	Means	Shown in the UI as
The step's key and its `label`	`multiply:` with `label: "Step 2 — Multiply"`	What this step does in the recipe	The title of the step's card, and the second name on its graph node
The Operation's `label` (under `run`)	`label: "Multiply two numbers"`	The app (OGC API Process) that does it	The app's name, everywhere: graphs, cards, the cost summary
The Operation's `id`	`id: multiply-two-numbers`	The process identifier the Cook calls	Only where a technical id is useful

The split matters because a user may swap the app in a step (section 5.2). The step stays the same ("Step 2 — Multiply"), but the app changes ("Multiply three numbers"): the Registry rewrites the step's Operation, and with it its label. An early version of the UI showed whatever name was nearest at hand, so one step appeared as its key on one screen, its app on another, and the title from the remote process description on a third. Now the Registry passes the Operation's label to every client (`process_label` in the DAG), and every screen follows one rule:

- **App name:** the Operation's `label` in the recipe; else the name of our own tool; else the `title` in the app's OGC API Process description; else its process id.
- **Step name:** the step's `label`; else its key.

A recipe author is therefore in control of every name a user sees. A step without a `label` shows its raw key (for example `varidian-cook_subsidence-analysis`), so we recommend giving every step a short label.

`format` beyond File inputs. CWL defines `format` only for File parameters. Our Cook also reads it on an Operation's non-File inputs and outputs: an input is sent as an OGC qualified value (`{"value": …, "mediaType": …}`), and an output is requested in that media type. In `playground-2`, Buffer's GML output is passed to GetArea this way. Strict CWL ignores these with a warning, so this use only means something to our Cook.

5.2 Extensions we needed

CWL describes *what* to run, not *where*. Three extension fields fill the gaps a recipe has in an App Store. They live in a namespace each recipe declares:

```
None
$namespaces:
  nldt: https://fi-group.github.io/nldt-tb2/ns#
```

Field	Where	Why it is needed
`nldt:processLink` (`rel`, `type`, `href`, `internalHref`)	On a step's `run`	Binds an abstract Operation to a concrete OGC API Process. The Cook calls `internalHref`; `href` is for people reading the recipe
`nldt:compatibleTools`	Top level, `{stepId: [toolId, …]}`	The recipe owner's allowlist of alternative tools a hub may swap in for a step
`nldt:jsonSchema`	On a workflow input, *optional*	The exact JSON Schema of a parameter when CWL types can't express it

Two addresses in `nldt:processLink`: `href` and `internalHref`. A process link carries two addresses for the same process:

- `href` is the **public** address, the one anyone can open in a browser. For our own tools it is under `https://nldt-tb2.clearly.app/api/tools/…`; for an external process it is that server's own address.
- `internalHref` is the address **the Cook calls**. For an external process it is the same public address. For our own tools it is their address on the stack's private Docker network (for example `http://tool-add:8000/processes/add-two-numbers`), which works only from inside that network.

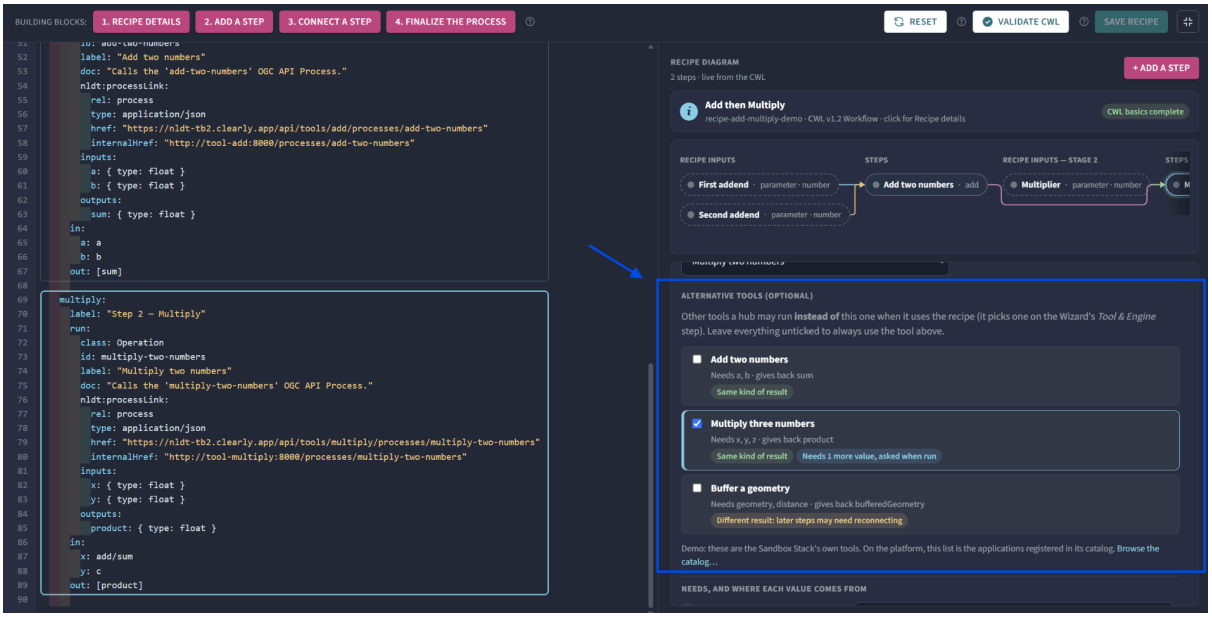
The second address exists because the Cook runs on the same private network as our own tools. Calling them there is direct: the request does not leave the server, and does not depend on the public gateway, its TLS certificate or the Cloudflare proxy in front of it. It is not a development shortcut; the deployed Cook uses it on every run.

Still, it is a deployment detail that has leaked into the recipe. A recipe should say *which* process a step uses, not how one particular installation reaches it: `http://tool-add:8000` means nothing to anyone reading the recipe, or to another cook running the same recipe elsewhere. A cleaner design keeps only the public `href` in the recipe and lets each cook map known public addresses to faster internal ones in its own configuration, the way it already does for data (`LOCATION_REWRITES`, section 5.3). We recommend that change (section 12).

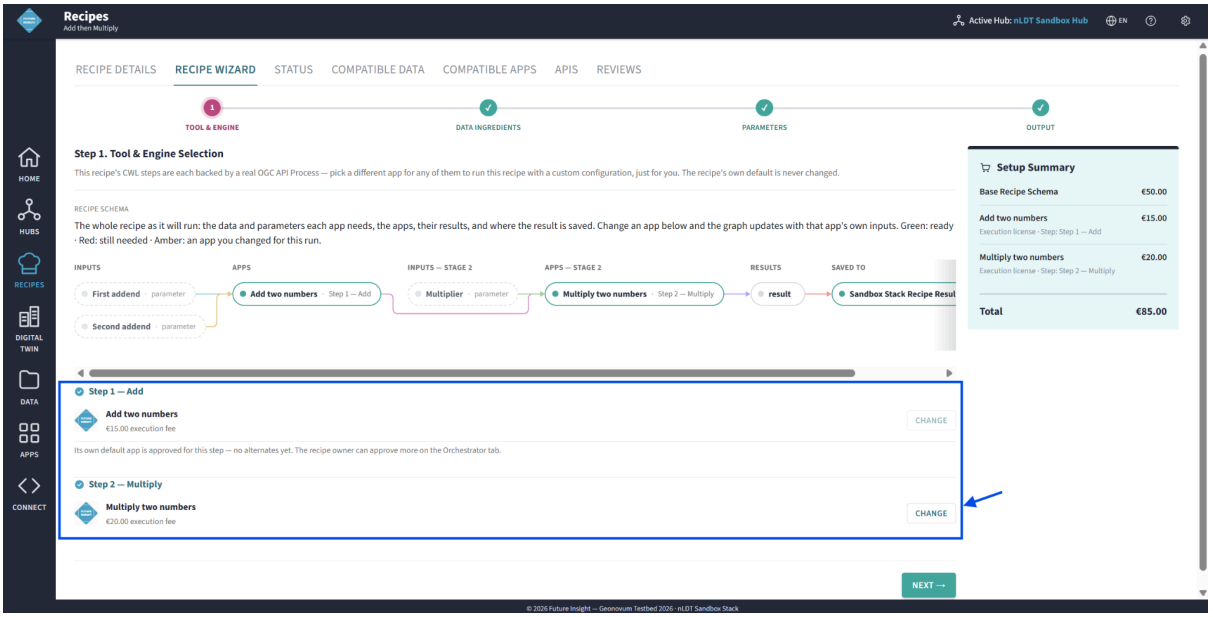
From `x-` to `nldt:`. Our first version used OpenAPI-style names: `x-process-link`, `x-compatible-tools`, `x-json-schema`. CWL allows extension fields only with a prefix declared in `$namespaces`, and `cwltool --validate` rejected every bundled recipe because it read the `x-` fields' contents as CWL. After renaming to a declared `nldt:` namespace, all five bundled recipes pass the reference validator, `cwltool --validate` (*cwltool 3.3.20260925135507*), and the derived schemas and DAGs are unchanged.

Two harmless warnings remain: cwltool still resolves `type` keys inside our fields, and reads roofer's process ids (`roofer:validate_point_cloud:v1`) as an undeclared prefix. The Registry now rejects the old `x-` names with an error naming the replacement. The namespace IRI does not resolve yet (404), which CWL does not require.

Running a recipe vs swapping tools. Running a recipe as written needs no central process registry: each step carries its own `nldt:processLink`. Tool swaps are more limited:



- Alternatives are checked against a built-in catalogue of our four tools (add, multiply, multiply3, buffer). Unknown ids in the allowlist are silently dropped, so a hub cannot swap in an external process such as roofer or ZOO-Project.
- A step without an allowlist entry allows only its own tool.
- A swap remaps inputs by position: the new tool's first input gets the old tool's first input's wiring, and so on. Inputs the new tool adds become new workflow inputs. This works for `multiply → multiply3`, but two tools whose inputs share an order but not a meaning would be wired wrongly without any error.
- The catalogue exists in four copies (Registry, Cook fallback, and two in the UI) that must stay in sync.
- Hand-editing `nldt:processLink` can point a step at any process; only the Wizard's swaps are allowlisted.



5.3 Data inputs vs parameters

A recipe declares a data input with standard CWL, not an extension: `type: File` (or `File[]`) plus a `format`. Every other input is a parameter.

- The value is always a reference: `{"class": "File", "location": "https://…", "format": "…/application/geo+json"}`.
- The Cook passes it on as an OGC by-reference input, `{"href": …, "type": …}`. The process fetches the data itself.
- Where a process expects its own shape, the recipe adapts the File with `valueFrom` on that step. Roofer, for example, wants `[{"kind": "url", "url": …}]`.
- The Registry lists data inputs as `data_inputs` in the DAG. The Wizard builds its Data Ingredients step from that, so any recipe, including one just edited, shows its data needs.

Location rewriting. A data input's location is the public Hub Datasets address the browser picked, under `https://nldt-tb2.clearly.app/api/hub/…` For our own tools only, the Cook swaps that public prefix for Hub Datasets' address on the private network (`LOCATION_REWRITES`), for the same reason as `internalHref` (section 5.2). External processes always receive the public address, and fetch the data from there; this only works because Hub Datasets is publicly reachable.

We considered and rejected two alternatives. A new `x-data-ingredient` field would duplicate a concept CWL already has. Inferring data inputs from the process description is impossible: roofer describes its point-cloud input as a LAS/LAZ URL only in prose.

5.4 Parameters: CWL types vs JSON Schema

OGC API Processes describe every input in JSON Schema; CWL has its own, weaker type system. For most inputs the CWL type is enough and the Registry derives the JSON Schema itself.

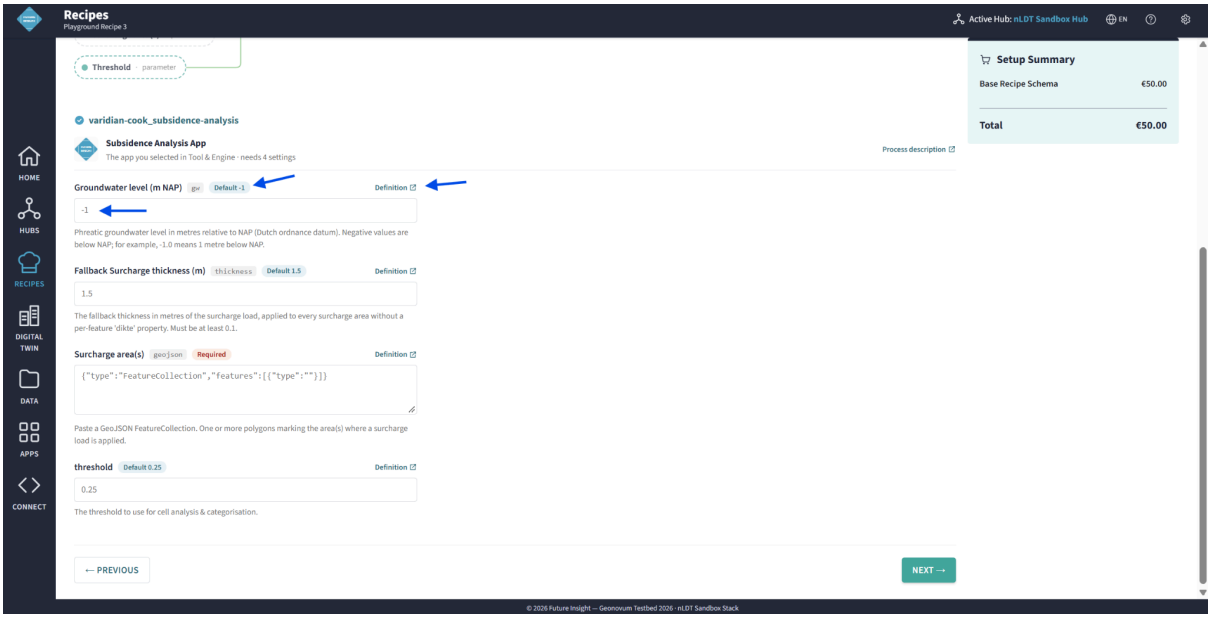
Process input (JSON Schema)	**Recipe input (CWL)**	`nldt:jsonSchema` **needed?**
`{type: number}`	`float`	No
`{type: string, enum: [gpkg, cityjson]}`	`enum`	No
A GeoJSON or LAZ file	`File` + `format`	No
`{oneOf: [...]}` with `const`, `pattern`, `minItems` (roofer's `bag`)	Only `Any`	Yes

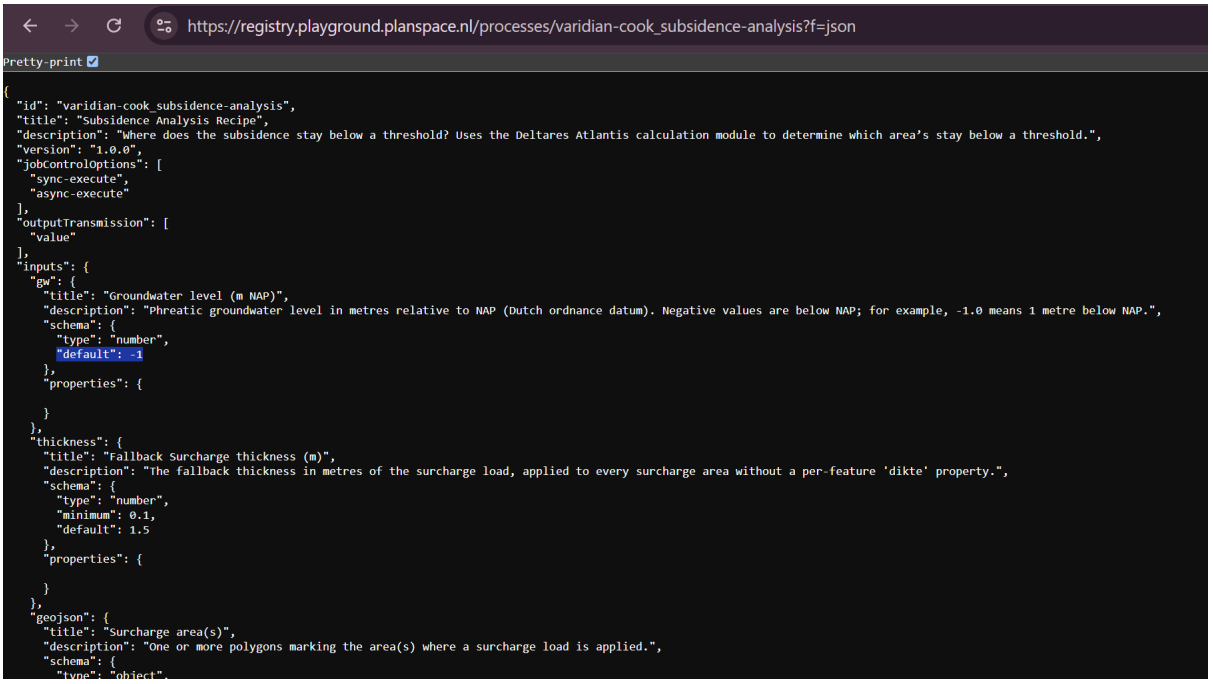
`nldt:jsonSchema` is descriptive only: the client builds its form from it, the Cook does not validate against it. Its weakness is that it is a copy pasted from the process description, so it goes stale if the process changes. Referencing the process's own schema would be better.

The process description fills the gaps, at display time. Most recipes say little about their parameters: a CWL type and perhaps a label. The app behind each step usually says much more, in a standard place: an OGC API Process description gives every input a `title`, a `description` and a JSON Schema, and JSON Schema has standard keywords for a `default` and for limits such as `minimum`. Varidian's subsidence process, for example, publishes `gw` with default -1.0, `thickness` with default 1.5 and minimum 0.1, and `threshold` with default 0.25.

So the Wizard fetches the description of each step's process (once per session, through the Registry's `fetch-description` proxy for external servers) and, for each recipe parameter, reads the input it is wired to. Two rules keep this safe:

- **The recipe always wins.** A label, description or default written in the CWL is used as written. The process only fills what the recipe leaves out.
- **Nothing is written back.** The process's information is shown, and its defaults are sent when a field is left empty, but the stored recipe never changes. If the process changes its description, the next Wizard session shows the new one, so this cannot go stale the way a pasted `nldt:jsonSchema` can.





This works for any recipe and any process, not only the ones in this testbed. Its limit is what process providers publish: the subsidence process gives no default or example for its GeoJSON input, so that field stays required, and the form can only show the expected shape (an empty FeatureCollection). A JSON Schema `example` entry would let the form show a real polygon (section 13).

5.5 Example

The bundled Rotterdam recipe, with comments and `doc` text removed:

```
None
cwlVersion: v1.2
class: Workflow
id: recipe-rotterdam-buffer-demo
label: "Rotterdam Boundary Buffer"
$namespaces:
  iana: https://www.iana.org/assignments/media-types/
  nldt: https://fi-group.github.io/nldt-tb2/ns#
inputs:
  geometry:                          # DATA input
    type: File
    format: iana:application/geo+json
  bufferDistance:                    # PARAMETER
    type: float
outputs:
  bufferedOutline:
    type: Any
    outputSource: buffer/bufferedGeometry
```

```
steps:
  buffer:
    label: "Step 1 — Buffer"          # the step's name
    run:
      class: Operation
      id: buffer-geometry             # OGC API Process id
      label: "Buffer a geometry"      # the app's name
      nldt:processLink:               # extension: where it lives
        rel: process
        type: application/json
                                                                    href:
"https://nldt-tb2.clearly.app/api/tools/geo-buffer/processes/buffer-geometry"
        internalHref: "http://tool-geo-buffer:8000/processes/buffer-geometry"
      inputs:
        geometry: { type: File, format: "iana:application/geo+json" }
        distance: { type: float }
      outputs:
        bufferedGeometry: { type: Any }
    in:
      geometry: geometry
      distance: bufferDistance
    out: [bufferedGeometry]
```

Appendix C gives the three playground recipes in full, with notes, including the roofer recipe's `valueFrom` chaining.

6. The Cookbook Registry

The Cookbook Registry is the tender's cookbook: it stores each recipe's raw CWL and derives, on every request, its input JSON Schema and execution DAG.

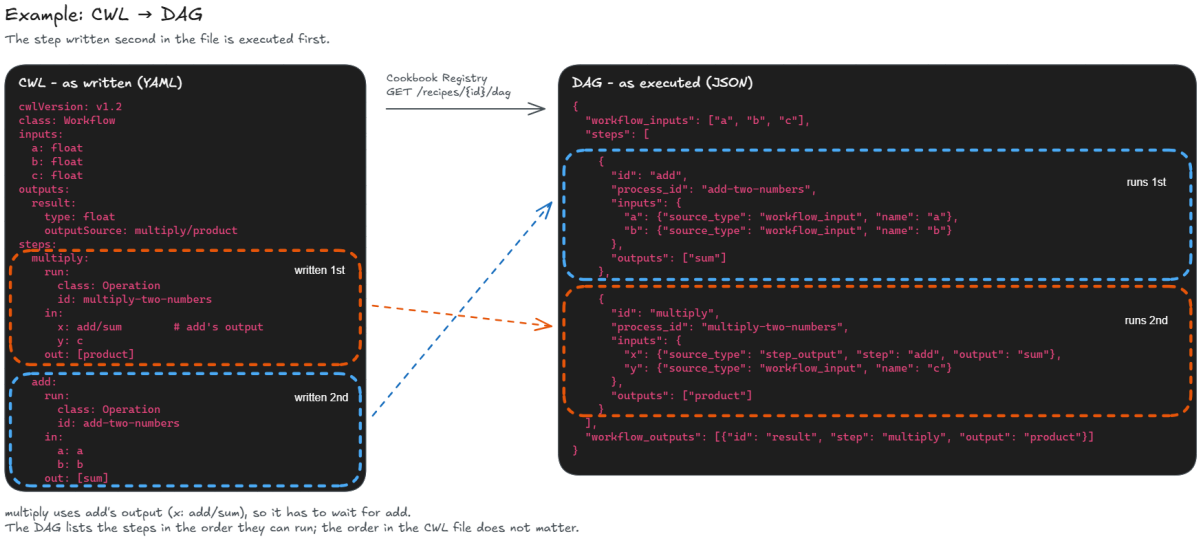
The tender suggests building the cookbook with OGC Building Blocks. We did not package the Registry's API as a Building Block: it is a small, purpose-built REST API and no existing block describes a recipe registry. The better candidates are the recipe format itself and the catalogue record. The `nldt:` CWL profile (section 5) and the recipe record (section 8) each have a schema and examples, which is what a Building Block contains. Publishing them through the OGC Building Blocks register is future work (section 12).

Endpoint	Purpose
`GET /recipes`	List recipes (id, title, description, keywords, timestamps)
`GET /recipes/{id}`	The CWL, verbatim (`application/cwl+yaml`)
`GET /recipes/{id}/original`	The bundled, known-good CWL, used by the editor's Reset
`GET /recipes/{id}/schema`	Derived JSON Schema of the recipe's inputs
`GET /recipes/{id}/dag`	Derived execution graph: steps in topological order, resolved inputs, `data_inputs` and `workflow_outputs`
`POST /recipes/{id}/effective`	Schema and DAG with a hub's `toolOverrides` applied; never stored
`POST /recipes/validate`	Checks a CWL document without storing it; always 200 with `valid: true/false`
`POST /recipes`	Validates, stores (upsert), and publishes a catalogue record
`DELETE /recipes/{id}`	Removes the recipe and retracts its record

Endpoint	Purpose
`GET /processes/fetch-description?url=…`	Fetches any OGC API Process description server-side for the guided builder, avoiding browser CORS limits. No allowlist or SSRF protection

Recipes are stored in SQLite in a Docker volume. Schema and DAG are derived fresh on every request, never cached.

CWL vs DAG. CWL is a document written for people: order in the file is arbitrary and it carries labels, docs and extensions. The DAG is the machine form: steps in topological order, each input resolved to a workflow input or another step's output. Section 4.2 explains why every part of the stack works from the DAG.



6.1 What the DAG contains

Field	Contents
`steps`	Every step, in an order in which each step comes after the steps it depends on

Field	Contents
`steps[].id`, `label`	The step's key and its name (its CWL `label`, else its key)
`steps[].process_id`, `process_label`	The OGC API Process the step calls, and the app's name (its Operation's `label`)
`steps[].process_link`	The step's `nldt:processLink`: the process's public address (`href`) and the one the Cook calls (`internalHref`, section 5.2)
`steps[].inputs`	For each input of the process: `{source_type: "workflow_input", name}` or `{source_type: "step_output", step, output}`, plus any `valueFrom` reshaping as a field `path` or object `template`
`steps[].outputs`	The outputs the step declares
`steps[].input_formats`, `output_formats`	Media types from CWL `format`, used for by-reference and qualified values (section 7.1)
`workflow_inputs`	The recipe's inputs
`input_defaults`, `optional_inputs`	What the Cook does with an input nobody sent: use its default, or leave it out

Field	Contents
`workflow_outputs`	What the recipe returns, and which step output each one is
`data_inputs`	The inputs that are datasets (CWL `File` + `format`), with their media types
`compatible_tools`	The owner's allowlist of alternative apps per step (`nldt:compatibleTools`)

The DAG of `recipe-add-multiply-demo`, shortened:

```json
{
  "workflow_inputs": ["a", "b", "c"],
  "steps": [
    {
      "id": "add",
      "label": "Step 1 — Add",
      "process_id": "add-two-numbers",
      "process_label": "Add two numbers",
      "process_link": {
                                                                "href":
"https://nldt-tb2.clearly.app/api/tools/add/processes/add-two-numbers",
        "internalHref": "http://tool-add:8000/processes/add-two-numbers"
      },
      "inputs": {
        "a": { "source_type": "workflow_input", "name": "a" },
        "b": { "source_type": "workflow_input", "name": "b" }
      },
      "outputs": ["sum"]
    },
    {
      "id": "multiply",
      "label": "Step 2 — Multiply",
      "process_id": "multiply-two-numbers",
      "process_label": "Multiply two numbers",
      "process_link": {
```

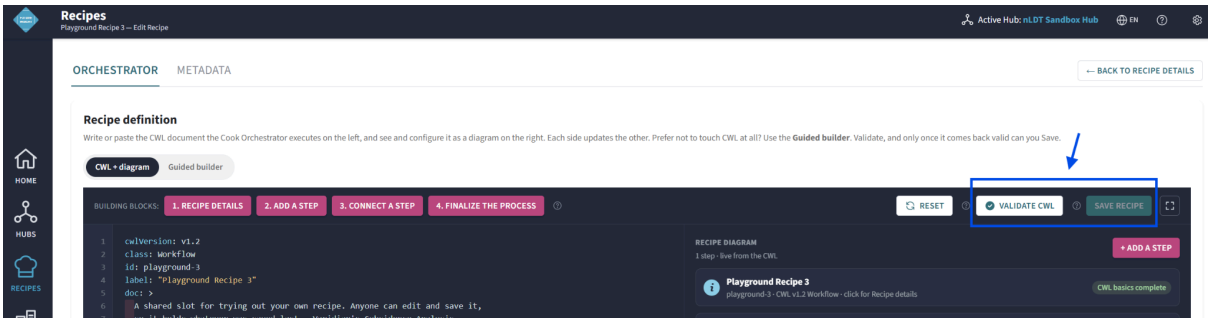
```
                                                                    "href":
"https://nldt-tb2.clearly.app/api/tools/multiply/processes/multiply-two-numbers
",
                                                                "internalHref":
"http://tool-multiply:8000/processes/multiply-two-numbers"
      },
      "inputs": {
        "x": { "source_type": "step_output", "step": "add", "output": "sum" },
        "y": { "source_type": "workflow_input", "name": "c" }
      },
      "outputs": ["product"]
    }
  ],
   "workflow_outputs": [{ "id": "result", "source_type": "step_output", "step":
"multiply", "output": "product" }],
       "compatible_tools":    {    "add":    ["add-two-numbers"],    "multiply":
["multiply-two-numbers", "multiply-three-numbers"] },
  "data_inputs": {}
}
```


Reading it: `multiply`'s input `x` is `add`'s output `sum`, so `multiply` must wait for `add`; its input `y` is the recipe input `c`. The recipe's `result` is `multiply`'s `product`. Even if the CWL listed `multiply` before `add`, the DAG would list `add` first.

6.2 Design choices

Two design choices matter:

- **Validation is a gate, twice.** The editor's *Validate* calls the validate endpoint; *Save* re-validates server-side. Nothing is stored unless a DAG can be built. Validation is structural only: it never calls the processes.



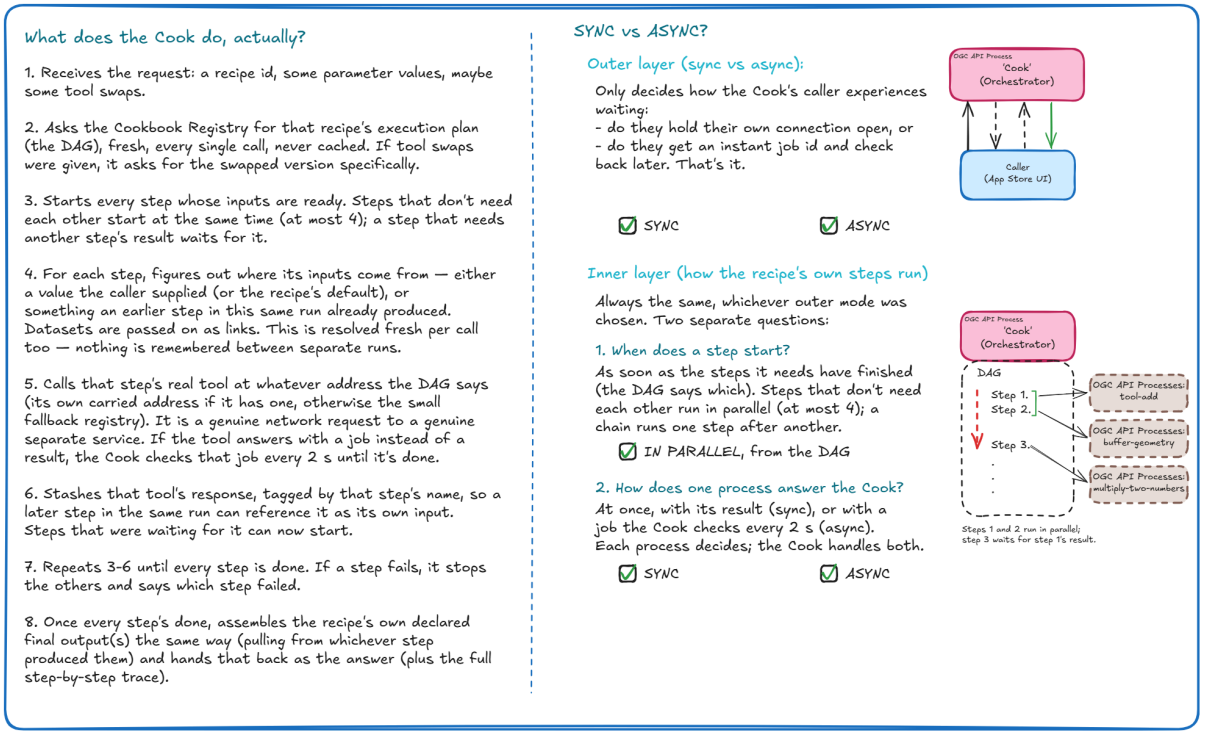
- **Catalogue outages don't lose recipes.** A recipe is stored first, then published. If the catalogue is down, the response says `catalog_published: false` plus the error, rather than failing.

One weakness is that if a stored recipe no longer parses (for example an old one still using `x-` names), `/schema` and `/dag` return *HTTP 500* because the parse error isn't caught. Re-seeding after a redeploy replaces stored recipes.

7. The Cook Orchestrator

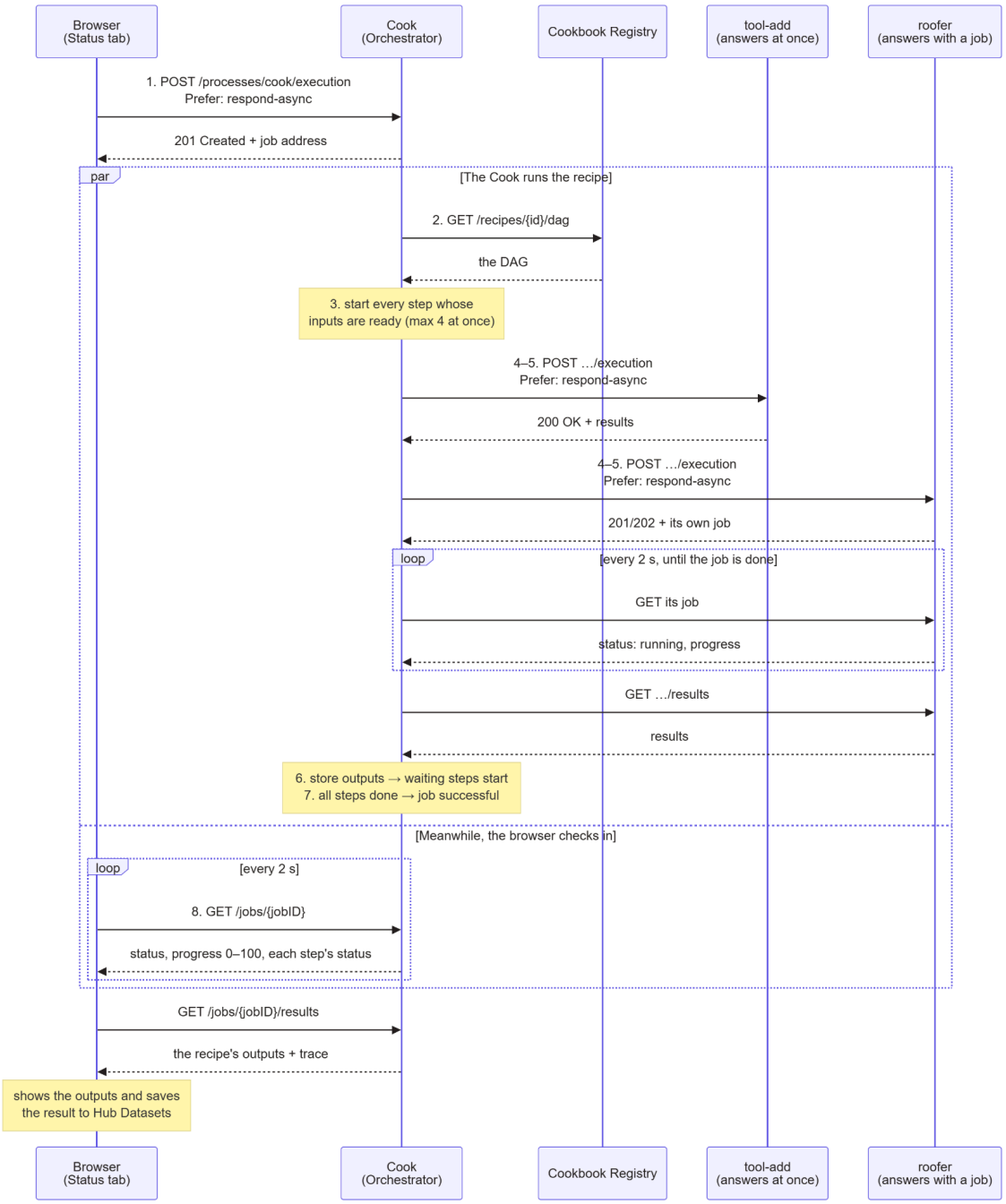
The Cook is itself an OGC API Processes Part 1: Core 1.0 service with exactly one process, `cook`, so any OGC API Processes client can execute a recipe without knowing anything about CWL. It is non-agentic: it walks the DAG and decides nothing.

The `cook` process takes `recipeId`, `parameters` and `toolOverrides` (`{stepId: toolId}`) and returns `result`. It declares the conformance classes `core`, `json`, `oas30`, `job-list` and `dismiss`.



7.1 What happens when you click 'Execute'

In short, the browser hands the recipe to the Cook and gets a job number back at once. The Cook then works through the recipe's DAG on its own: it starts every step whose inputs are ready, calls that step's OGC API Process, waits for it (following the process's own job when the process runs asynchronously), and passes its outputs on to the steps that need them. Meanwhile the browser asks the Cook every 2 seconds how far it is.



In detail:

1. In the browser (Status tab). Execute collects three things:

- **Parameters:** each value entered in the Wizard; for an empty field, its default (the recipe's own, else the app's, see section 5.4).
- **Data ingredients:** each dataset picked in the Wizard, as a CWL File, `{"class": "File", "location": "<URL of the dataset>", "format": "<media type>"}`. The location is a link; the data itself is never sent.
- **App swaps:** `toolOverrides, {stepId: toolId}`, if the hub swapped an app for this run.

It posts these to the Cook, `POST /processes/cook/execution` with the header `Prefer: respond-async` and the body `{"inputs": {"recipeId": …, "parameters": {…}, "toolOverrides": {…}}}`. The Cook answers immediately with `201 Created`, a `Location` header with the job's address (`https://nldt-tb2.clearly.app/api/orchestrator/jobs/{jobID}`) and the job's first status (`accepted`). Execute is disabled while a required parameter or data ingredient is still missing.

2. In the Cook: get the plan. The job runs on its own background thread, so the Cook can take other requests meanwhile. It asks the Registry for the recipe's DAG (`GET /recipes/{id}/dag`), or, if the hub swapped an app, for the effective DAG (`POST /recipes/{id}/effective`).

3. In the Cook: start every step that is ready. A step is ready when every step it takes input from has finished. Ready steps start at once, each on its own worker thread, at most 4 at a time (`COOK_MAX_PARALLEL`). Steps that do not depend on each other therefore run in parallel; a chain still runs one step after another. In `playground-2`, for example, Hello World (Python), Hello World (JavaScript), Centroid and Buffer all start together, and GetArea starts the moment Buffer has finished.

4. In the Cook: prepare one step's request. For each input of the step's process, the DAG says where its value comes from:

- **From a recipe input:** the value the browser sent; if none, the recipe's CWL `default`; if none and the input is optional, it is left out; otherwise the run fails with "Missing required parameter".
- **From another step's output:** the value that step returned.
- **Reshaped**, if the recipe's CWL has a `valueFrom` for it: a field picked out of the value, or a new object built from it. This is how a recipe hands data to a process in that process's own shape, for example roofer's `[{"kind": "url", "url": …}]`.
- **Data inputs** (CWL Files) become OGC by-reference inputs, `{"href": "<location>", "type": "<media type>"}`: the process downloads the data itself. For our own tools on the internal Docker network, the Cook first swaps the public address for the internal one (`LOCATION_REWRITES`).
- **A `format` on a non-File input** wraps the value as an OGC qualified value, `{"value": …, "mediaType": …}`. A `format` on an output asks the process for that output in that media type.

5. In the Cook: call the process. The Cook posts `{"inputs": {…}}` to the step's process, at its `nldt:processLink.internalHref` plus `/execution`, again with `Prefer: respond-async`. A process can answer in one of two ways, and the Cook handles both:

- **Synchronously:** `200 OK` with the results in the response. Our own four tools always do this; they ignore the header.
- **Asynchronously:** `201` or `202` with a job of its own. Roofer, the Geolabs ZOO-Project processes and Varidian's subsidence process support this. The Cook finds the job's address (the `Location` header, else the job's `self` or `monitor` link, else `/jobs/{jobID}`), then asks it for its status every 2 seconds, copying the remote status, progress and message into the step's own progress. When the job reports `successful`, the Cook fetches `…/results`. When it reports `failed`, the Cook fetches the error document from the same address, because some servers (ZOO-Project) leave their status message at "accepted" even after failing.

6. In the Cook: read the outputs. Processes return results in different shapes: our own tools as `{"outputs": {…}}`, standard OGC results as `{name: value}` or `{name: {"value": …}}`, ZOO-Project as multipart text when a process has several outputs, and some servers as the bare value of a single-output process. The Cook turns each of these into one shape, `{outputName: value}`, and stores it. Every step waiting for this step's outputs may now be ready (back to point 3). The Cook also records a trace entry: exactly what the step was sent, what it returned, and whether its process was found through its `nldt:processLink` or the fallback table (section 7.3).

7. In the Cook: finish, or stop on the first failure. When all steps are done, the Cook picks out the recipe's outputs (`workflow_outputs`) and marks the job `successful`. If a step fails, the Cook starts no new steps, stops the ones still running, dismisses their remote jobs, and marks the job `failed`, saying where the failure came from (section 7.4). Time limits apply to each HTTP call, each remote job and the whole run (section 7.2).

8. Back in the browser. While the Cook works, the browser asks `GET /jobs/{jobID}` every 2 seconds. The answer has the overall status, a `progress` from 0 to 100 (finished steps count fully, a running step by its remote job's own progress if it reports one), a one-line message such as "*Step 2 of 3: Reconstruct buildings — running*", and the status of every step. The Status tab shows this as a list of steps. A Cancel button sends `DELETE /jobs/{jobID}`; the Cook stops between steps, or within 2 seconds while waiting on a remote job, and dismisses that job too.

When the job is `successful`, the browser fetches `GET /jobs/{jobID}/results`: the recipe's outputs plus the trace. It shows each output (a GeoJSON output as a summary, since it may be large) and immediately saves the result to the Sandbox Stack Recipe Results dataset in Hub Datasets, named after the recipe with the date and time. If any output is GeoJSON, that output is saved, so it can be shown on a map; otherwise the whole result is saved as JSON. Section 9.4 describes where saved results can be viewed.

The Cook is non-agentic throughout: it never decides which process to call or how to combine results. Everything it does follows from the DAG.

7.2 Sync and async

There are two layers. The **outer layer** is how the caller experiences waiting: sync (hold the connection) or async (`Prefer: respond-async`, get a job, poll `/jobs/{id}`). The **inner layer** is how the Cook calls each step: it always asks for asynchronous execution (`Prefer: respond-async`). A process that answers synchronously anyway is used as-is; one that returns a job is polled. Our own four tools always answer synchronously.

Sync is the protocol default (a caller that sends no `Prefer` header) and returns `{result, trace}` in one response; the App Store UI always uses async (section 7.1). Async returns 201 with a `Location` header; job status includes per-step progress, a 0–100 `progress` and a message. Each asynchronous cook job runs on its own background thread, so the Cook accepts new jobs while others are running. The job store is in memory, so a restart loses jobs.

Async is mandatory in practice. Roofer's building reconstruction outlives any HTTP or proxy timeout. Remote jobs are polled every 2 s, and the time limits are configurable:

Setting	Applies to	Default	In docker-compose
`TOOL_REQUEST_TIMEOUT`	Each HTTP call	10 s	90 s
`TOOL_JOB_TIMEOUT`	Each remote job	300 s	—
`TOOL_JOB_POLL_INTERVAL`	Polling a remote job	2 s	—
`COOK_TIMEOUT`	The whole run	900 s	—
`COOK_MAX_PARALLEL`	Steps running at once	4	—

When a job times out or is cancelled, the remote job is dismissed (best-effort `DELETE /jobs/{id}`). Dismissing a cook job stops it between steps, or within one poll interval while it waits on a remote job.

7.3 Where a step runs

A step names its process by an abstract id. The Cook resolves it to a URL in this order: the step's own `nldt:processLink.internalHref`, then a static fallback table (`TOOL_REGISTRY`) for the four built-in tools. `href` is never called; section 5.2 explains why there are two addresses, and why we would drop the internal one. The fallback keeps older recipes working; there is no dynamic discovery service.

We tested the vendor-agnostic claim through the real UI in an early build (early September): replacing the multiply step's process with a divide process, without code changes, returned `(2 + 3) / 4 = 1.25`, with the trace showing the new process was called. The divide tool was later replaced by `multiply3`; today the Wizard offers the equivalent swap, multiply → multiply3, on `recipe-add-multiply-demo`.

7.4 Where a failure came from

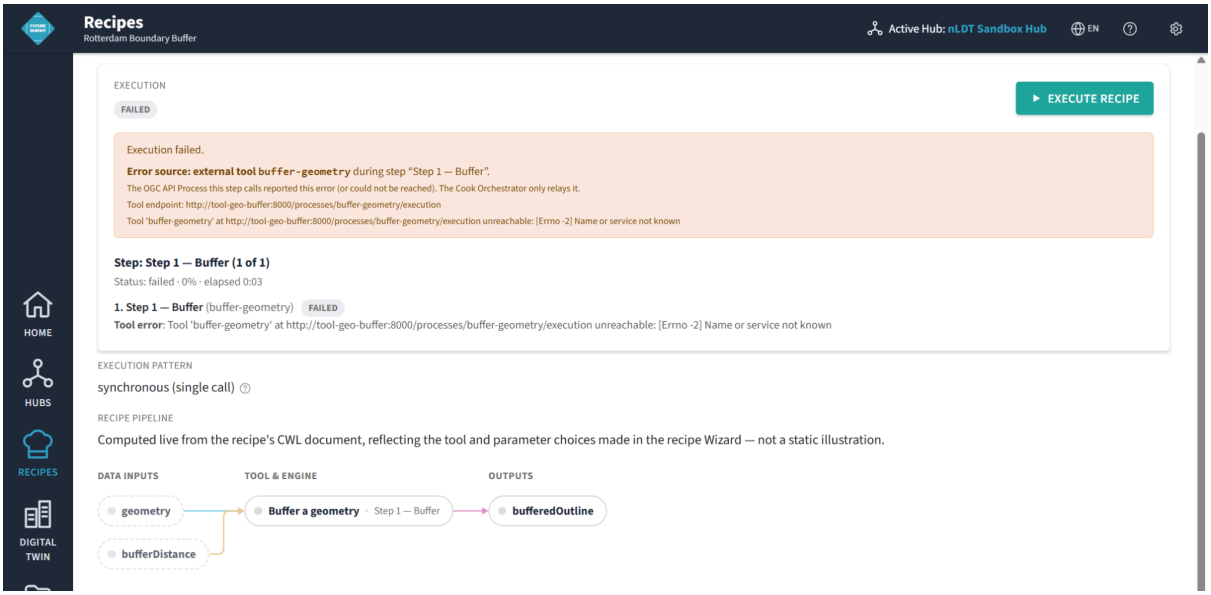
A recipe calls processes the user does not control, so a failed run must say whose failure it is. A failed cook job's status carries an `errorSource` field, and so does each failed step. It has one of three values:

- `tool`: the step's OGC API Process returned an error status, its job failed or timed out, it could not be reached, or its response could not be read. The field also names the process id, its URL and the step.
- `registry`: the Cookbook Registry could not be reached or did not know the recipe.
- `cook`: the Cook itself: a missing parameter, the overall time limit, a configuration problem or a bug.

The Status tab uses this field to tell the user "*Error source: external tool `buffer-geometry` during step 'Buffer'*" or "*Error source: Cook Orchestrator (this platform)*". Failed steps in the step list start with "*Tool error:" or "Cook Orchestrator error:*".

Two cases can be labelled wrongly. A tool response the Cook cannot parse counts as a tool error, even if the Cook's parsing is at fault. A network problem between the Cook and a tool also counts as a tool error, although the message says "unreachable". `errorSource` is a field beyond OGC API Processes' `statusInfo`; synchronous execution still returns only the error text.

Figure below shows the banner for a run whose tool could not be reached. It was made on a local copy of the stack, with the buffer tool's container stopped, so the live services and recipes were not touched.



8. Catalogue and recipe metadata

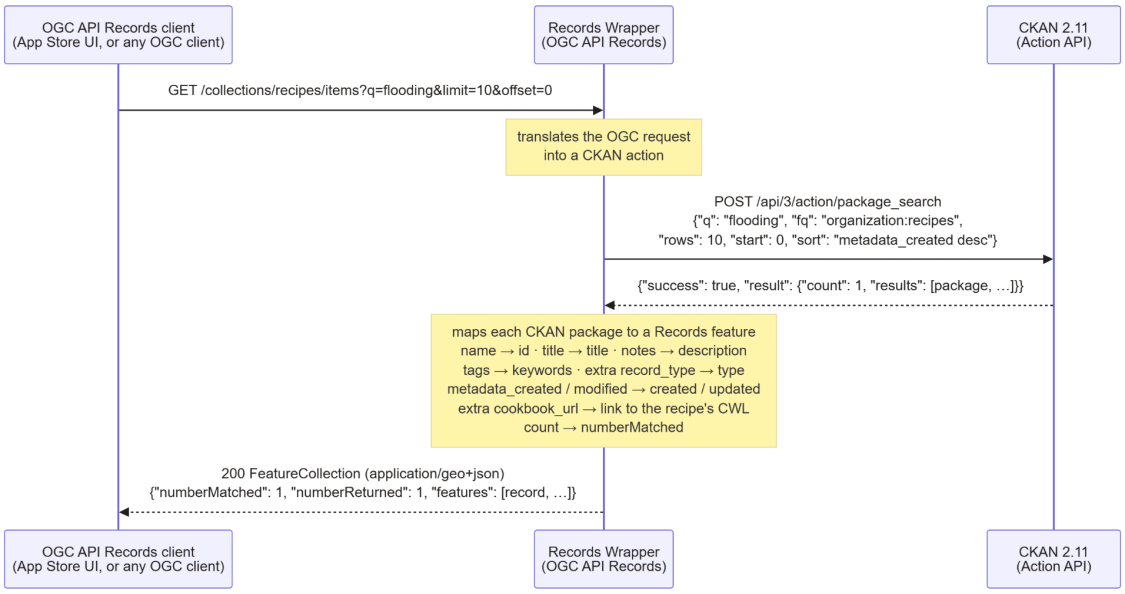
The catalogue implements OGC API Records Part 1: Core over a real CKAN 2.11 instance. Recipe metadata in DCAT-AP-NL is not implemented; section 8.3 gives a first proposal.

8.1 Records Wrapper over CKAN

The Records Wrapper exposes the landing page, conformance, collections, items (parameters `q`, `limit` and `offset` only) and a single item, with one collection, `recipes`, mapped to one CKAN organisation. It translates each OGC request into CKAN Action API calls (`package_search`, `package_show`, …) and maps the results back to Records features.

Records Part 1 is read-only, so publishing uses two non-standard write endpoints, `POST` and `DELETE` on `/collections/{id}/items`, which the Registry calls to publish and retract records.

Figure below shows one OGC API Records search, `items?q=flooding`, and the CKAN `package_search` call the Records Wrapper turns it into.



We first built our own SQLite store, then switched to real CKAN to match the tender's architecture. The swap touched only the wrapper's repository module; the API and its clients were unchanged.

Lessons from running CKAN:

- `ckan-base` 2.9 and 2.10 images ship a uWSGI without plugins and fail to start; 2.11 works.
- CKAN initialises its datastore even when the plugin is not listed, so it needs a real database.
- CKAN soft-deletes packages, so re-publishing a deleted id must revive it rather than create it.
- CKAN already has a `type` field; our record type is stored as the extra `record_type`.

8.2 What a record contains

A record carries discovery metadata only: title, description, keywords, timestamps, `type: recipe`, and a link to the CWL in the Cookbook Registry. The CWL itself lives only in the Registry: the catalogue describes the thing, but it does not hold it.

The link uses the relation `…/rel/ogc/1.0/recipe-source` with media type `application/cwl+yaml`. That relation sits in OGC's namespace but is not registered there. A registered relation (`describedby`, or `alternate` with the CWL media type) or one in our own namespace would be correct.

Two gaps in the records today:

- The link's `href` is built from `COOKBOOK_PUBLIC_URL`, which was set to the Docker-internal `http://cookbook-registry:8000`, so an outside client could not follow it. The gateway setup now sets the public address (`https://nldt-tb2.clearly.app/api/cookbook`), and records carry it from the moment a recipe is stored again, as happened for the bundled recipes after the redeployment of 29 October.
- Keywords are empty for all bundled recipes, because the seed script sets none.

8.3 Recipe metadata in DCAT-AP-NL: what is missing, and what to add

The catalogue we built carries discovery metadata only (section 8.2); we did not implement a DCAT-AP-NL profile for recipes. But building and running recipes end to end showed clearly what such metadata has to say. This section sets that out: what DCAT-AP-NL already offers, what a recipe needs that it does not cover, and what we propose to add.

What DCAT-AP-NL already offers. DCAT-AP-NL is the Dutch profile of DCAT-AP, itself a profile of W3C DCAT. It describes catalogues and what is in them: **datasets**, their **distributions** (a downloadable or accessible form of the data, with a media type and an access address), and **data services** (an API, with an endpoint address and an endpoint description). Around these it has the usual descriptive metadata: title, description, keywords, theme, spatial and temporal coverage, publisher, contact point, licence, access rights, conformance to a standard (`dct:conformsTo`), version, dates, and general relations between resources.

That covers much of a recipe's *discovery* metadata. A recipe has a title, a description, keywords, a theme, an area it is meant for, a publisher and a licence, and all of those fit. It also covers the recipe's *definition* as a file: the CWL document can be a distribution, with its media type and the standards it conforms to.

What a recipe needs that DCAT-AP-NL does not cover. A recipe is neither data nor a running service: it is an executable plan that combines both. Everything about how it runs, what it needs and what it produces has no place in DCAT-AP-NL today. Each row below comes from something the testbed showed:

A recipe needs to say	Why (section / finding)	In DCAT-AP-NL today	What to add
That it is a recipe	It is neither a dataset nor a data service	No class for it	A class `Recipe`, a subclass of `dcat:Resource`,

A recipe needs to say	Why (section / finding)	In DCAT-AP-NL today	What to add
			catalogued next to datasets and services
Where its executable definition is, and in which format	The CWL is the recipe (sections 5, 8.2)	Partly: a distribution with a media type and `dct:conformsTo`	Use a distribution for the CWL, conforming to CWL v1.2 and to a published recipe profile (the `nldt:` extensions). Register a media type for CWL, which IANA does not have
Which apps it uses, one per step, by name	A recipe *is* a chain of apps (sections 5.1, 6.1)	Partly: a data service can describe one API endpoint	Steps: for each step its name and the data service (OGC API Process) it calls, including the app's name and the address of its process description
In which order the steps run, and which step feeds which	The DAG (sections 4.2, 6.1)	No	Either the step dependencies, or simply a pointer to the definition; we recommend the pointer, so the CWL stays the single source and the metadata cannot contradict it

A recipe needs to say	Why (section / finding)	In DCAT-AP-NL today	What to add
Which alternative apps may replace a step's app	`nldt:compatibleTools` (section 5.2)	No	Per step, the allowed alternative data services
Which datasets it needs, in which format	Data inputs (section 5.3, finding 11)	No: a media type exists only on a distribution	Data inputs: name, description, required or optional, accepted media types, and optionally example or recommended datasets from the catalogue
Which parameters it takes, with their type, default and limits	Parameters (sections 5.4, 9.3, finding 17)	No	Parameters: name, title, description, required or optional, default, and a JSON Schema, by reference to the app's own input schema rather than a copy
What it produces, in which format	Outputs (section 6.1)	No	Outputs: name, description, media type
How it runs	Sync and async, duration (section 7.2, finding 12)	No	Expected execution mode (synchronous or job-based) and typical duration

A recipe needs to say	Why (section / finding)	In DCAT-AP-NL today	What to add
Whether its apps need a login or a key	Authenticated apps (findings 23, 24)	Only `dct:accessRights`, as free text or a public/restricted code	Per app: whether access is needed, from which identity provider, and whether access can be delegated to a cook
What a run costs	Commercial apps (finding 25)	No; DCAT allows an ODRL policy on a distribution, which could carry terms	Per app and per recipe: the price model (per run, subscription, free), possibly as an ODRL policy
Who may change it, and which version this is	Shared, unversioned editing (finding 21, section 3)	Partly: version and dates exist, rights to edit do not	Version, version history, owner; editing rights belong in the catalogue's access control rather than in metadata
Where a result came from	Saved results (section 9.4)	No; PROV-O can express this but is not part of the profile	For a dataset produced by a recipe run: the recipe (and its version), the apps used and the input values, as provenance (`prov:wasGeneratedBy` a run that `prov:used` the recipe and its inputs)

A recipe needs to say	Why (section / finding)	In DCAT-AP-NL today	What to add
Who it is for, and in which project phase	Facets shown in the UI (section 9.2)	Theme and coverage yes; target roles and project phase no	Two facets: target roles and project phase, each with an agreed code list

Proposal. Extend DCAT-AP-NL with a small recipe module rather than bending datasets or data services to fit:

- A class **Recipe** (subclass of `dcat:Resource`) with the normal descriptive properties, plus links to its **steps**, **data inputs**, **parameters** and **outputs**, its expected **execution mode**, and its **access** and **cost** requirements.
- A **Step** names its role in the recipe and the **data service** (OGC API Process) it calls, plus any allowed alternatives. Each data service is described once, as DCAT-AP-NL already allows, with its endpoint and process description.
- A **Data input** and an **Output** each carry a name and the media types they accept or produce. DPROD's input and output ports are a candidate to reuse for these.
- A **Parameter** points at the app's own input schema instead of copying it.
- The CWL file is the recipe's distribution, conforming to CWL v1.2 and to the recipe profile.
- Results produced by a recipe run carry provenance back to the recipe.

Most of this metadata can be generated from the recipe itself: the Registry already derives the steps, their apps, the data inputs with media types, the parameters and the outputs (section 6.1), and the apps' process descriptions provide titles, descriptions and defaults (section 5.4). Only discovery facets (theme, target roles, phase), access, cost and ownership need to be written by people. That makes a recipe record cheap to keep accurate.

In CKAN, the extra fields would come from `ckanext-scheming` and the RDF output from `ckanext-dcat`. Neither is installed today; CKAN runs only the `envvars` and `datastore` plugins.

An illustrative record for `recipe-rotterdam-buffer-demo`, built from the proposal (not validated; the `nldt:` terms are placeholders for the module's own vocabulary):

```
None
@prefix dcat: <http://www.w3.org/ns/dcat#> .
@prefix dct:  <http://purl.org/dc/terms/> .
@prefix nldt: <https://fi-group.github.io/nldt-tb2/ns#> .
```

```
<https://nldt-tb2.clearly.app/api/records/collections/recipes/items/recipe-rotterdam-buffer-demo>
  a dcat:Resource, nldt:Recipe ;
  dct:identifier "recipe-rotterdam-buffer-demo" ;
  dct:title "Rotterdam Boundary Buffer"@en ;
    dct:description "Buffers Rotterdam's municipal outline, read live from PDOK."@en ;
  dcat:distribution [
    a dcat:Distribution ;
                                                    dcat:accessURL
<https://nldt-tb2.clearly.app/api/cookbook/recipes/recipe-rotterdam-buffer-demo> ;
                                                    dcat:mediaType
<https://www.iana.org/assignments/media-types/application/cwl+yaml>  ;   # not IANA-registered
    dct:conformsTo <https://w3id.org/cwl/v1.2/>, nldt:profile
  ] ;
  nldt:step [
    nldt:stepName "Step 1 — Buffer" ;
    nldt:usesService [
      a dcat:DataService ;
      dct:title "Buffer a geometry" ;
                                                  dcat:endpointURL
<https://nldt-tb2.clearly.app/api/tools/geo-buffer/processes/buffer-geometry/execution> ;
                                                dcat:endpointDescription
<https://nldt-tb2.clearly.app/api/tools/geo-buffer/processes/buffer-geometry>
    ]
  ] ;
        nldt:dataInput    [    nldt:name    "geometry"    ;    dcat:mediaType
<https://www.iana.org/assignments/media-types/application/geo+json> ] ;
   nldt:parameter  [  nldt:name  "bufferDistance"  ;  dct:title  "Buffer distance (meters)" ; nldt:required true ] ;
     nldt:output          [   nldt:name   "bufferedOutline"   ;   dcat:mediaType
<https://www.iana.org/assignments/media-types/application/geo+json> ] ;
  nldt:executionMode "synchronous" .
```

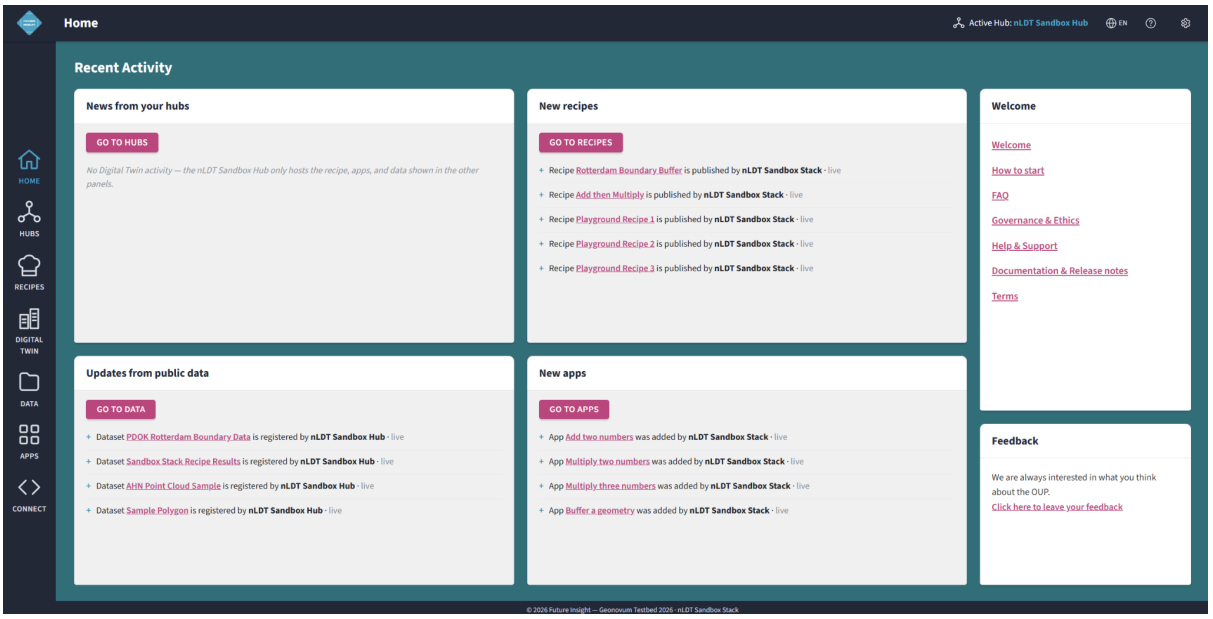
9. The client: App Store UI

The App Store UI finds recipes, binds them to a hub's own data and tools, and executes them. It is a static, no-build web app published to GitHub Pages and calls the services directly from the browser. Its aim is that someone who has never seen a CWL file or an OGC API Process description can still run a recipe and understand what it does.

Built in the image of Clearly Hub. The App Store UI follows the structure, styling and overall look and feel of Future Insight's own product, Clearly Hub, also called the Open

Urban Platform (OUP). The idea behind the OUP is a shared platform where organisations, each with its own *hub* (a municipality, for example), publish and combine data, apps and digital twins, and work on them together. The App Store's navigation (Home, Hubs, Recipes, Digital Twin, Data, Apps, Connect), its "active hub", its catalogue cards and its recipe pages all come from there.

We chose this deliberately. It shows recipes in the setting where they would actually be used, next to a hub's own data and apps, rather than in a bare test client. Parts of the OUP that are outside this research topic (reviews, hub management, digital twins, pricing) are shown as they would look, but are not functional in the sandbox.



9.1 How the UI uses the recipe

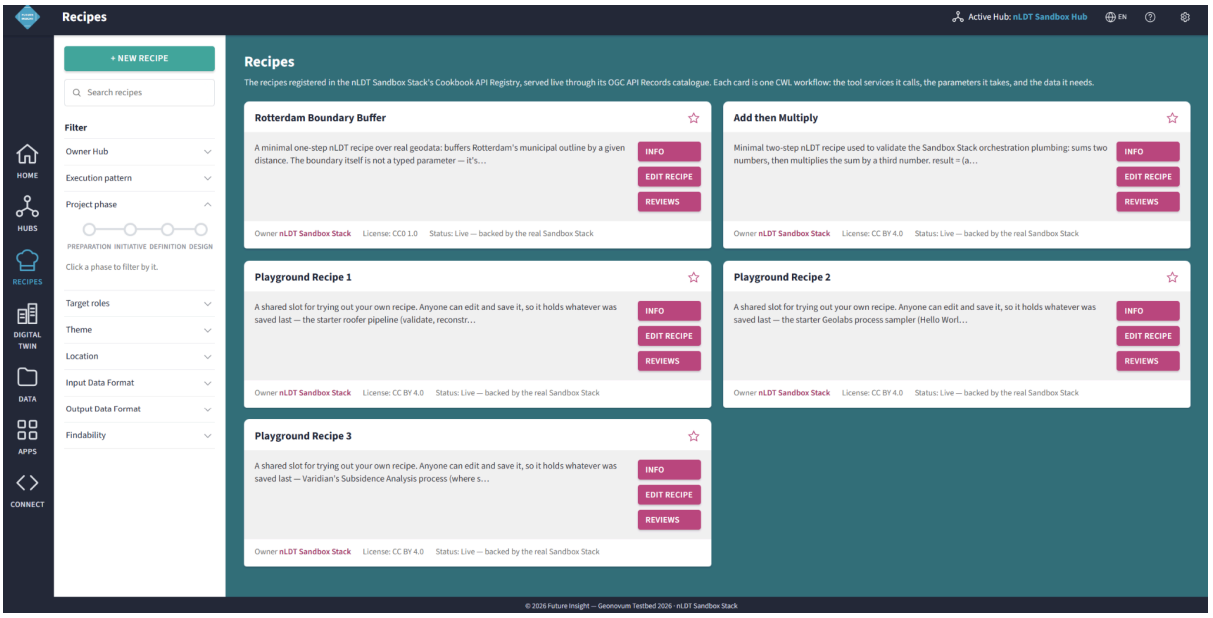
The UI never interprets CWL itself (section 4.2). Everything it shows about a recipe comes from the Registry's answer (DAG and input schema), from the catalogue record, or from the OGC API Process descriptions of the recipe's apps:

What the user sees	Where it comes from
Recipe title, description	The catalogue record (seeded from the CWL's `label` and `doc`)
The recipe graph (every Wizard step, Recipe Details, Status)	The DAG: inputs, steps and their wiring, outputs

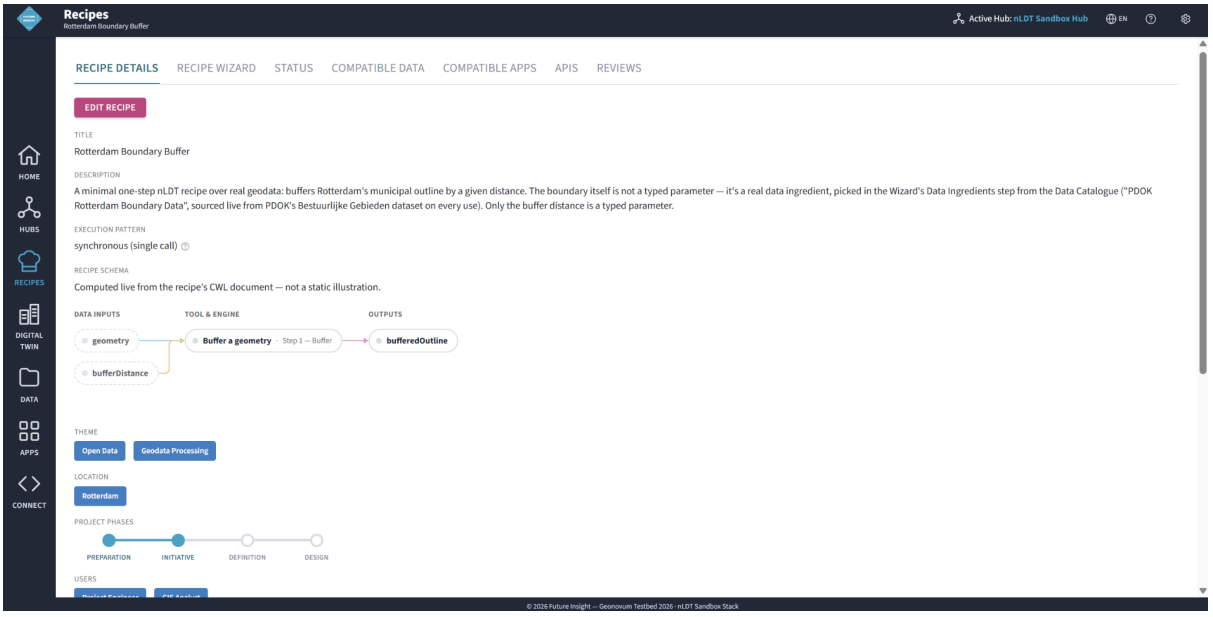
What the user sees	Where it comes from
A step's name	The CWL step `label` (`label` in the DAG)
An app's name	The CWL Operation `label` (`process_label` in the DAG), with fallbacks (section 5.1)
Which inputs need a dataset, and of which media type	`data_inputs` in the DAG (CWL `File` + `format`)
The parameter form: field types, required fields, defaults	The input schema (CWL types, `default`, `nldt:jsonSchema`)
Parameter titles, descriptions, limits and defaults the recipe doesn't give	The app's own OGC API Process description, read at display time (section 5.4)
Which alternative apps a step allows	`compatible_tools` in the DAG (`nldt:compatibleTools`)
A swapped app's own inputs	The effective DAG and schema (`POST …/effective`)
Links to an app's definition	The step's `nldt:processLink` (or our own tool's address)
Run progress, errors, result	The Cook's job status and results (section 7.1)

9.2 Catalogue, recipe pages and home page

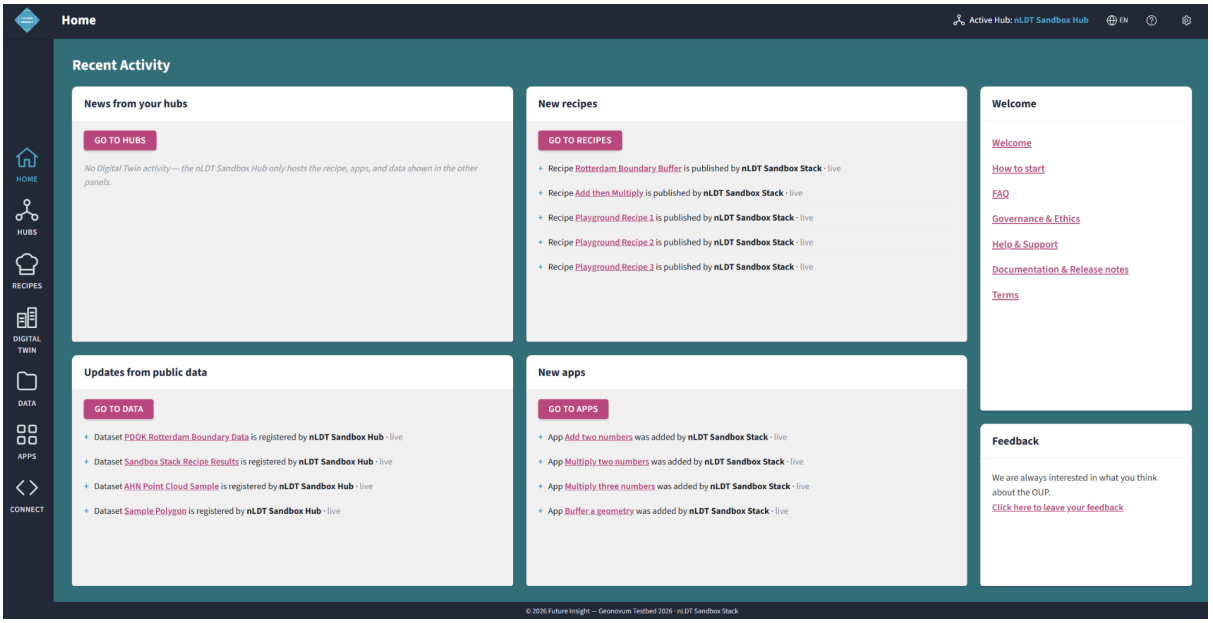
Catalogue. Recipes from the Records Wrapper, filterable by facets. Separate pages list Data (Hub Datasets) and Apps (live OGC API Processes, from each one's own process description).



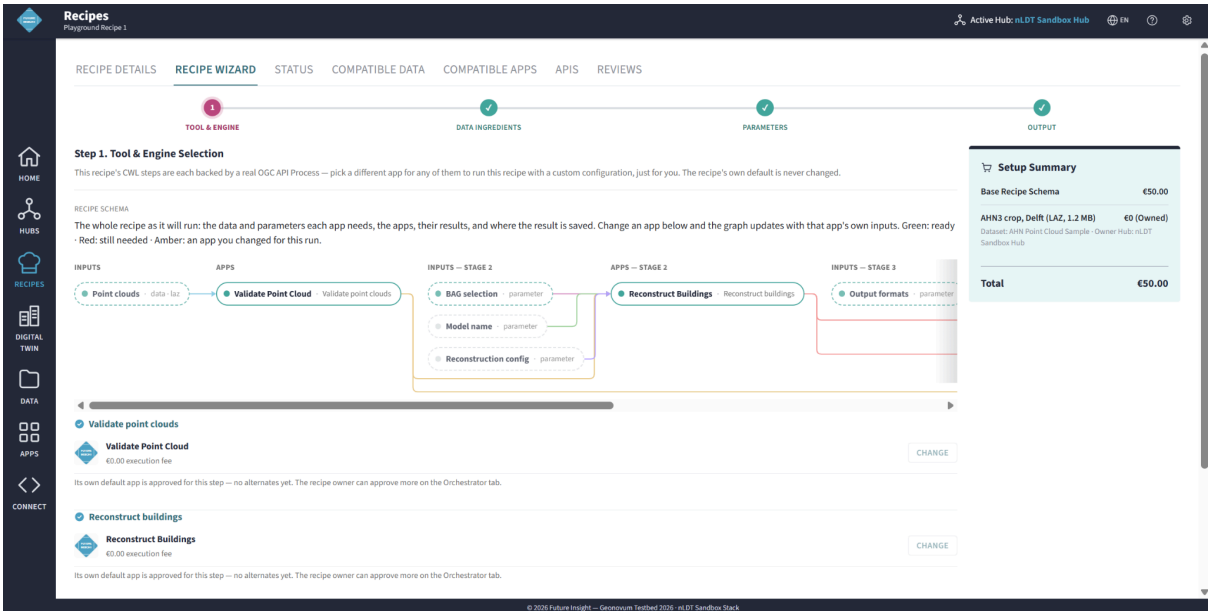
Recipe detail. Metadata, the recipe's graph, and its compatible data and apps. A banner on a live recipe points to the Recipe Wizard tab, where a run is set up.



Home page. Recent activity: the recipes, apps and datasets live in the sandbox.



9.3 The Recipe Wizard



The Wizard binds a recipe to a hub in four steps. Every step shows the same **recipe graph** above its own controls, so the user always sees the whole recipe while setting up one part of it:

- **Inputs:** each data input (marked "data · GeoJSON" and so on) and each parameter, by its readable title.
- **Apps:** the app of each step, named as in section 5.1. with the step's name beside it.

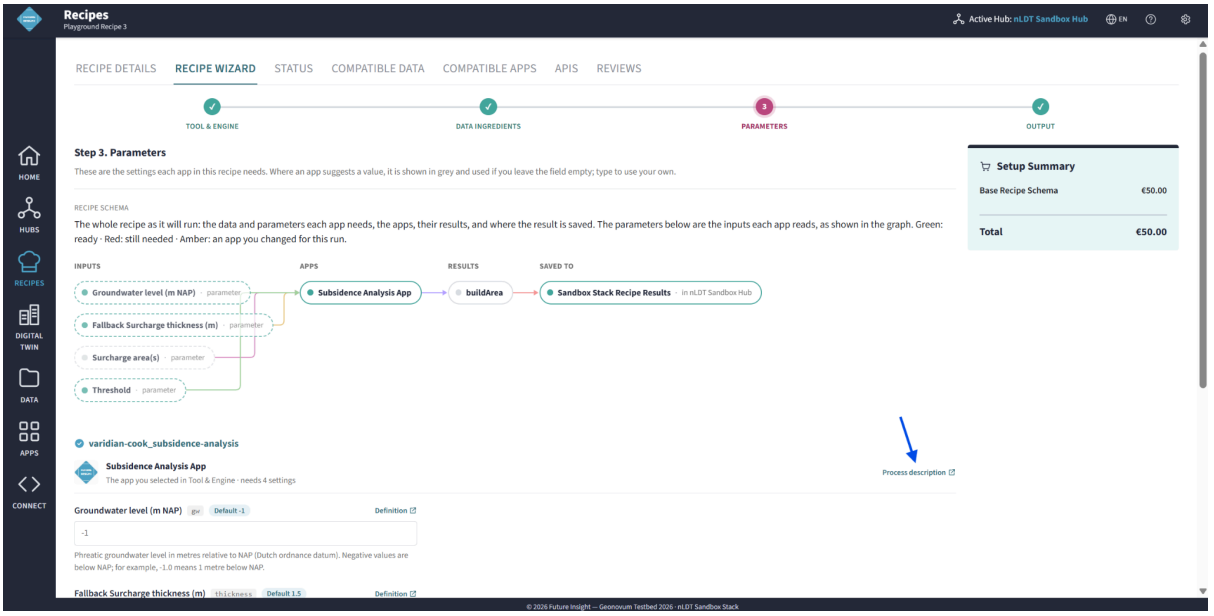
- **Results:** the recipe's outputs, always in their own column at the end, even when a step's output is also used by a later step.
- **Saved to:** where the result is stored.

Colours show the state of the setup: green for ready (a dataset picked, a value or default set), red for still needed, amber for an app swapped for this run. The graph is drawn from the effective DAG (section 4.2), so it always shows what the Cook will actually run.

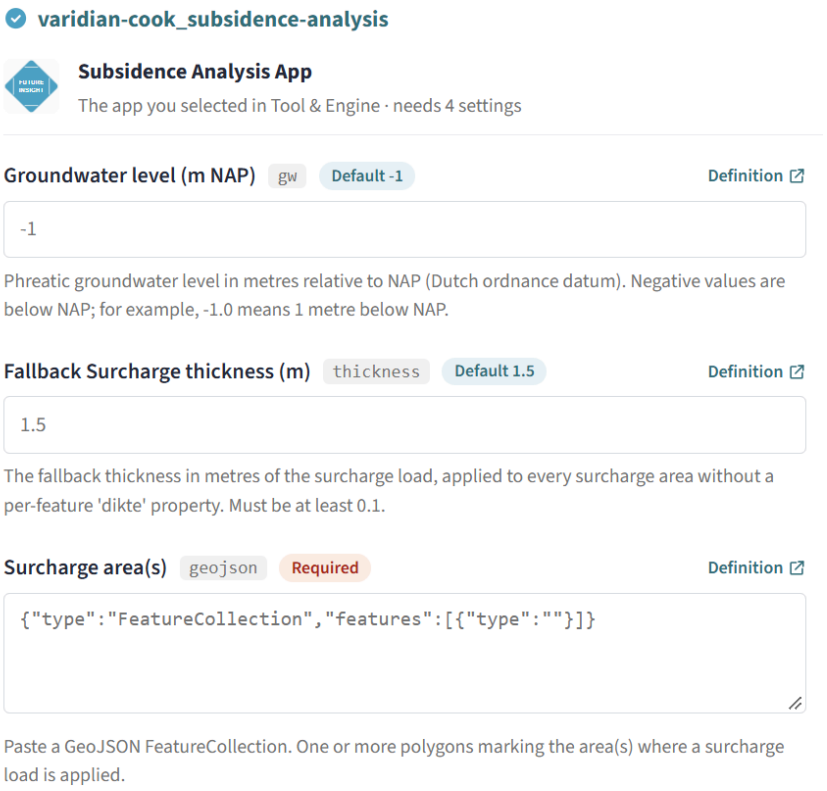


1. **Tool & Engine.** One card per step: the step's name, and the app that runs it. If the recipe owner allows alternatives (`nldt:compatibleTools`), "Change" offers them. Swapping an app asks the Registry for the effective recipe and redraws the graph straight away: swapping multiply for multiply3 on `recipe-add-multiply-demo` turns that app amber and adds its third input, `z`, to the graph and to the Parameters step. The swap applies to this run only; the stored recipe never changes.
2. **Data Ingredients.** For each `File` input, a dataset from Hub Datasets whose media type matches the input's `format`.
3. **Parameters.** The parameters are grouped by the app that reads them, so the form reads as "*this app needs these settings*" rather than as CWL wiring.

   Each group starts with the same app row as step 1, and a "Process description" link to the app's OGC API Process description (JSON).

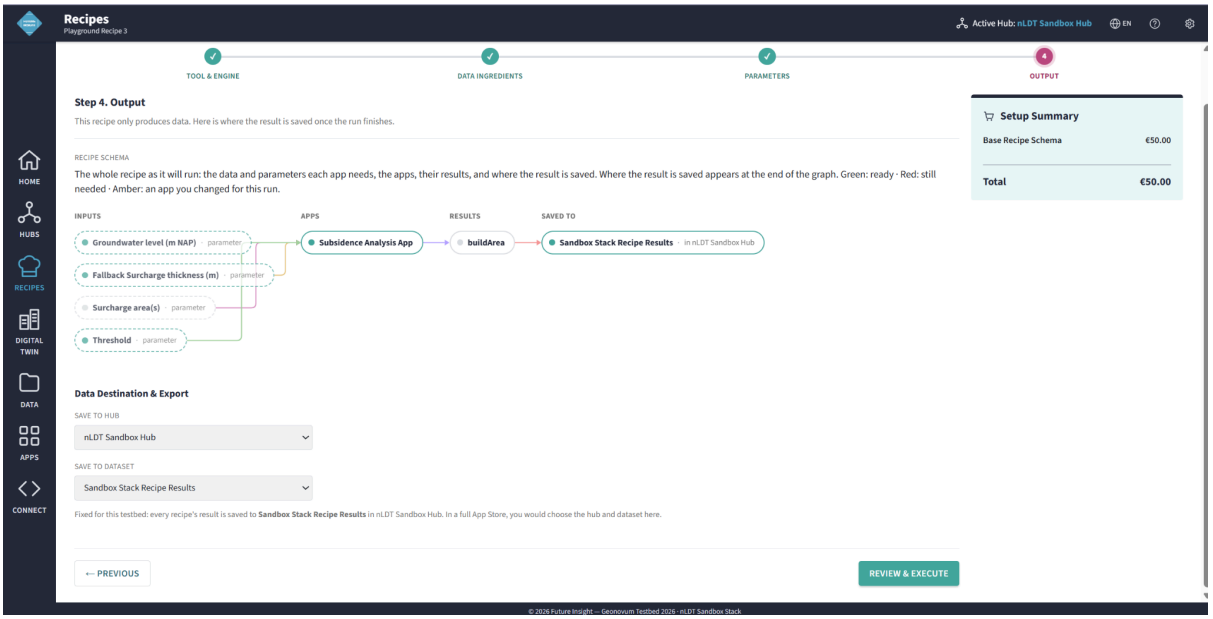


   Each field shows the parameter's readable title, its technical name as a small tag, a chip ("Default -1", "Required" or "Optional"), one line of help (its description and limits, such as "Must be at least 0.1"), and a "Definition" link to the same process description.



A default is shown as grey placeholder text: an untouched field sends it, typing replaces it, clearing the field brings it back. Structured values (objects, lists, GeoJSON) are typed as JSON in the same way, with the expected shape as placeholder; there are no special template buttons. Section 5.4 explains where titles, descriptions and defaults come from. A parameter used by two apps appears once, under the first, with "*Also used by …*"; one no app reads appears under "*Not used by any app*".

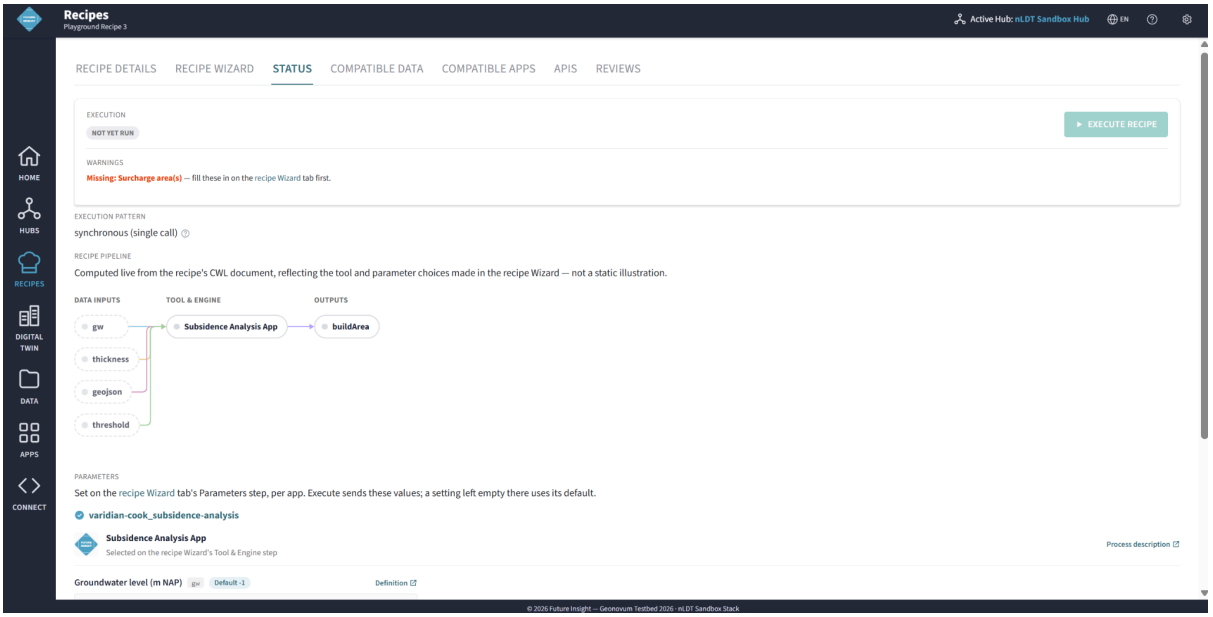
4. **Output.** Where the result is saved. In this testbed every result is saved to the Sandbox Stack Recipe Results dataset in the nLDT Sandbox Hub, so the hub and dataset dropdowns are shown, as they would be in a full App Store, but locked to that destination, with a note saying why.



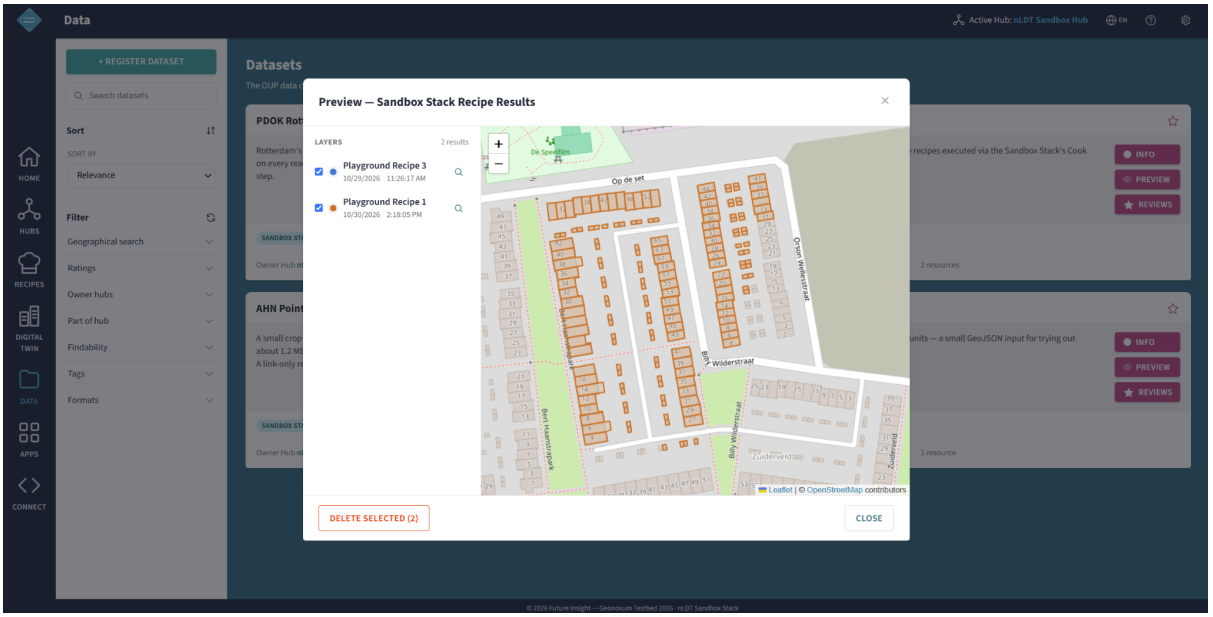
Every step can be opened at any time; nothing forces completing one step before the next. A setup summary beside the steps, styled as a basket, lists the apps used and an execution fee for each. The fees are illustrative and nothing is charged; it only shows where pricing would appear (finding 25).

9.4 Status tab and saved results

Status. Shows the recipe graph, the data ingredients, and the parameters grouped per app exactly as in the Wizard, read-only, each with the value that will be sent (marked "default" when it is one). It also works when the Wizard was never opened: missing values fall back to their defaults, the same way the Wizard does. Execute stays disabled, with a warning naming what is missing, until every required parameter and data ingredient has a value. Execute, progress, cancelling, the result and saving it are described in section 7.1; a failure says whose it is (section 7.4).



Saved results. Every successful run is saved to the Sandbox Stack Recipe Results dataset. Its Preview, on the Data page, shows all saved results on one map, each as a layer that can be switched on and off. The layer list shows each result by its recipe's name, with the date and time it was saved. A zoom button per result fits the map to that result and opens its label; a result with no geometry (for example the number from recipe-add-multiply-demo) says so instead.

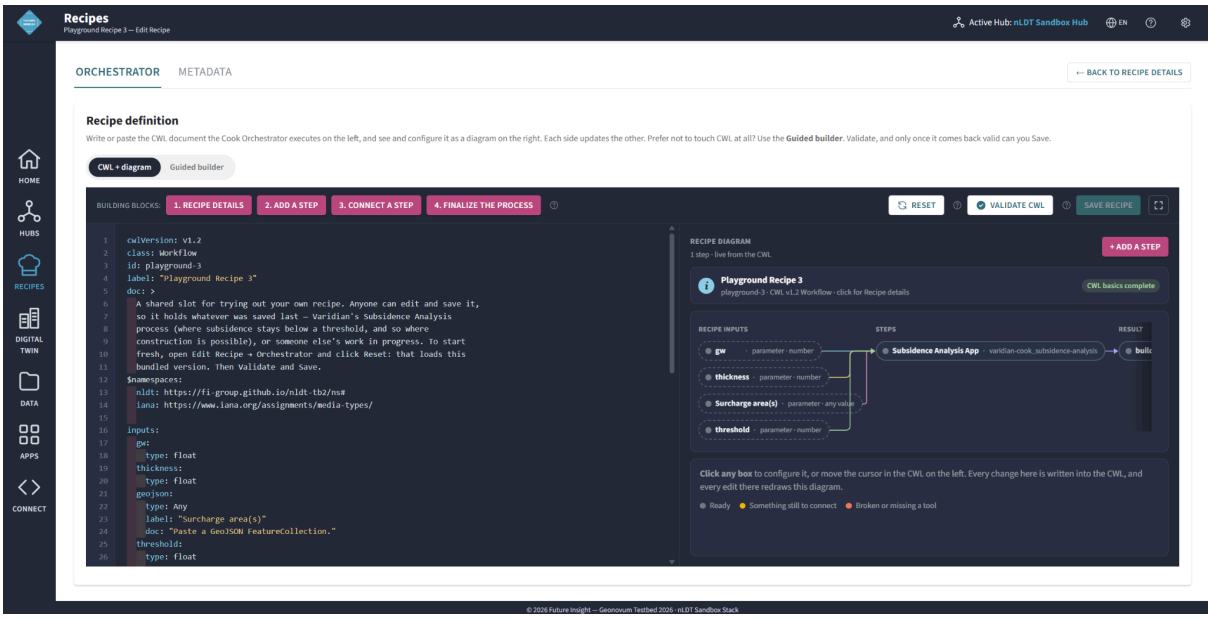


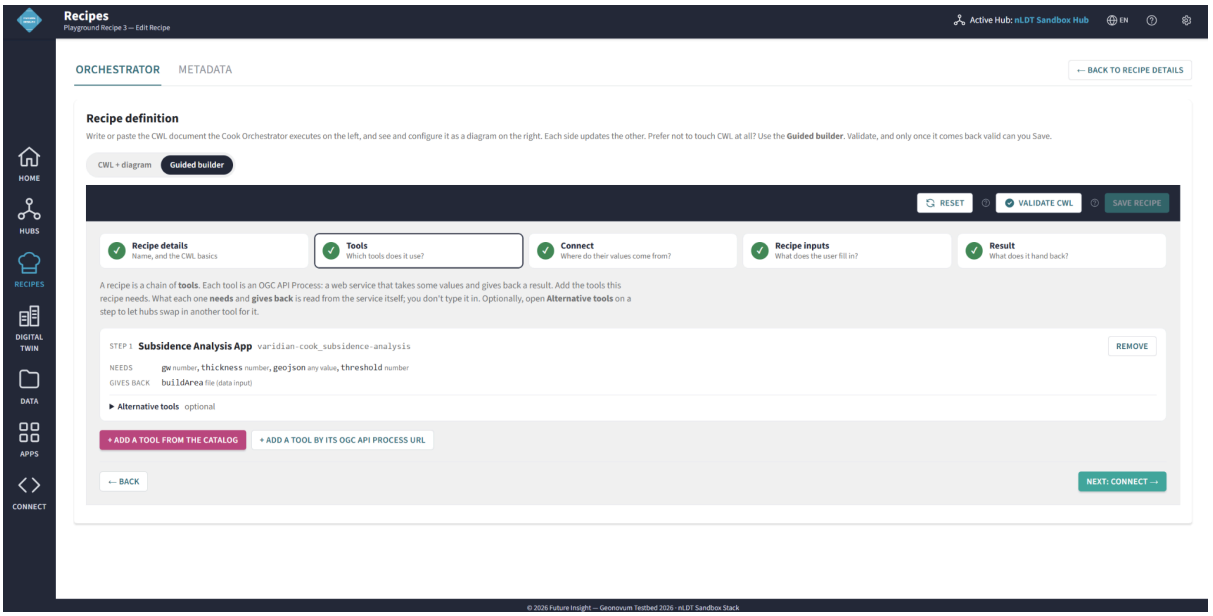
A result that links to a CityJSON model, such as roofer's reconstruction (artifacts.cityjson.href), is shown by its building footprints: the browser fetches the model, takes each building's LoD0 footprint, converts it from RD New (EPSG:28992) to WGS84, and draws it; clicking a building shows its id and its roof and ground heights. Only the footprints are drawn, since the map is 2D. "Delete selected" removes only the results whose layer is switched on, after a confirmation that lists each of them. Hub Datasets

delete them one by one (`DELETE /datasets/{id}/resources/{resourceId}`); built-in datasets such as the live PDOK boundary cannot be deleted.

9.5 Orchestrator editor

The recipe's CWL as syntax-highlighted text, as a diagram and as a guided builder. The builder adds a process by URL: it reads the process description and generates the step, its inputs and outputs, and its process link. It connects inputs and marks an input as data or parameter. It displays `valueFrom` reshaping but does not write it; that is still done by hand. *Save* is enabled only after a successful *Validate*, editing again disables it, and Save re-validates on the server. *Reset* reloads the bundled original.





The guided builder answers the main usability finding from the first plugfest: writing a recipe required knowing exactly how we had structured the CWL.

10. Plugfest and interoperability

At the first plugfest, we executed recipes against external processes from other participants. They ran end to end after small fixes.

What interoperated:

- Roofer staging (validate → reconstruct → convert), called by reference with a 1.2 MB crop of AHN3 tile c_37en2 (Delft), from TU Delft's 3dfier example data. Roofer staging returns fixture results and does not download the point cloud, so this proves the plumbing, not a reconstruction: whatever the input, it returns the same sample model, 165 buildings in Nijmegen.
- Geolabs ZOO-Project processes (Buffer, GetArea, Centroid), on the ZOO-Project server GeoLabs runs on Geonovum's testbed host, fetching a polygon by reference from a public URL.
- OGC API Processes clients that build URLs themselves can call our `cook` directly, since it is itself a process. Clients that follow the links in its answers got wrong addresses behind the gateway until this was fixed (see Conformance below).

What we learned from participants:

- Writing a recipe was not user-friendly: users had to know how we had structured the CWL. This led to the guided builder (section 9).
- A participant asked to parameterise the buffer recipe by city name instead of always using Rotterdam's outline. This is future work (section 12).
- A participant asked whether a client can read a process's inputs automatically when adding it as a step. The guided builder now does this from the process description.
- By-reference data must be reachable from the process, not just the browser. An external process can only fetch data at a public address, which is why Hub Datasets is published.

Conformance: OGC's official test suite. We tested every OGC API Processes service of ours, the Cook and the four tools, with OGC's own test suite for OGC API - Processes - Part 1: Core 1.0 (TEAM Engine with `ets-ogcapi-processes10` 1.3, the version OGC publishes as a Docker image). The suite runs 54 tests per service. We ran it three times: against the live stack before any fixes, against the fixed services on their own (and through a copy of the gateway), and against the live stack after the fixes were deployed during the final iterations..

Service	Live, before the fixes (passed / failed / skipped)	Live, after the fixes (29 Oct)	What still fails
Cook (`cook`)	18 / 31 / 5	**32 / 17 / 5**	17 execute tests, all using the suite's own test inputs
Add, multiply, multiply3 (one codebase)	13 / 27 / 14	**21 / 19 / 14**	18 execute tests, and the job list being empty
Buffer	13 / 28 / 13	**23 / 18 / 13**	17 execute tests, and the job list being empty

Skipped tests cover features the services do not offer (bounding-box or binary inputs, and, for the tools, asynchronous execution).

What the suite found, and what we fixed:

- **Errors were not in OGC's format.** Every error was FastAPI's default `{"detail": …}`; OGC requires an exception document with a `type`, and names the types for an unknown process, an unknown job and results that are not ready. All services now answer `{"type", "title", "status", "detail"}` with OGC's type URIs. (4 to 6 failures per service.)
- **No link to the API definition.** A landing page must link to its OpenAPI document (`service-desc`) or its documentation (`service-doc`). All services now link to both.
- **The tools' process list was incomplete:** it lacked the required top-level `links`. Fixed.
- **The Cook's job list** lacked top-level `links`, and a job's status contained `null` values (for example `started` before a job starts), which the schema does not allow. Fixed; a job's status now also links to itself (`self`, `monitor`) and, once done, to its results.
- **Results of an unfinished job** answered 409. The standard asks for 404 with `result-not-ready`; a failed job now answers with its own error. Fixed.
- **The tools had no job list at all.** They run synchronously and never have jobs, but the suite expects the endpoint; it now answers with an empty list. The suite still

marks this as failed because it wants at least one job, which a synchronous-only service cannot have.

- **Live only: links lost the gateway prefix.** The Cook's job `Location` header and its advertised links left out `/api/orchestrator` and used `http`. The suite followed them, got the App Store's HTML page instead of JSON, and failed 8 job tests. The cause: the gateway strips each service's `/api/<service>` prefix before passing a request on, so a service never sees it, and the services did not trust the gateway's "this came in over https" header. Fixed in the gateway setup: each service now starts with its public prefix (uvicorn `--root-path`) and trusts the gateway's forwarded headers. Live, the Cook now scores the same as on its own (32 passed). Our App Store UI was never affected, because it builds its own addresses.

What still fails, and why. Almost all remaining failures are the suite's *execute* tests. They expect the server to host OGC's test process, `echo`, which accepts every kind of input (text, numbers, objects, files by reference) and returns it unchanged. Our processes take specific inputs: two numbers, a geometry and a distance, or a recipe id. So the suite's test values are rejected, as they should be (the Cook, for example, is asked to run the recipe `"teststring"`), and these tests cannot pass. They test the server's execution machinery with a process it does not have, not whether our processes behave correctly. Hosting a small `echo` process next to the Cook would let the suite test the Cook's job handling end to end (section 12).

Conformance: the Geonovum OGC checker. We also ran Geonovum's OGC checker (`@geonovum/ogc-checker` 1.3.2, the command-line version of https://geonovum.github.io/ogc-checker/), on the live OpenAPI documents of all six services: the Cook and the four tools against OGC API - Processes 1.0, and the Records Wrapper against OGC API - Records 1.0. Unlike OGC's test suite, this checker does not call the service. It reads only the service's OpenAPI document (`/openapi.json`) and checks that the document describes what the standard requires.

First result: it could not read our documents. The checker accepts OpenAPI 3.0 only, and FastAPI, the framework all our services use, generates OpenAPI 3.1. Every service got a single "unrecognized format" warning, and nothing was checked. This confirms a risk listed below: our services declare the `oas30` conformance class, and their landing pages label the document as 3.0, but they serve 3.1.

Second result, on copies converted to 3.0. To see what the checker would find otherwise, we converted copies of the documents to OpenAPI 3.0 and ran it again. The checker reported only errors, no warnings:

Service	Errors	Missing from the document only	Real gaps	Not applicable
Cook (`cook`)	19	11	8	0
Each tool (add, multiply, multiply3, buffer)	17	5	3	9
Records Wrapper (catalogue)	3	0	3	0

- **Missing from the document only.** FastAPI writes into the document only what the code declares. So the documents give no response schema for the landing page, conformance, process list, process description, job status or job list. They also leave out the `201` answer to an asynchronous start, and the `404` answers for an unknown process, an unknown job and results that are not ready. The services do give all of these answers, as OGC's test suite confirmed above. A client that builds itself from the document would not know about them.
- **Real gaps.** The process list has no `limit` parameter, which Core requires. The Cook declares the job-list conformance class, but its job list ignores the filters that class requires (`limit`, `type`, `processID`, `status`, `datetime`, `minDuration`, `maxDuration`). The tools have no `/jobs/{jobID}` or `/jobs/{jobID}/results`, which Core requires even of a service that only runs synchronously. The catalogue answers with `application/json`, where Records requires `application/geo+json` for records and `application/ogc-catalog+json` for the collection.
- **Not applicable.** The checker applies the job-list rules, and the rule for asynchronous execution, to the tools as well, although the tools claim neither.

Other risks, from reading the code (not covered by the Processes test suite):

- **OpenAPI version.** The services declare `oas30`, but FastAPI serves OpenAPI 3.1 at `/openapi.json`. Confirmed by the Geonovum checker, which cannot read 3.1 (see above).

- **Result shape.** Sync execution and `GET /jobs/{id}/results` return `{result, trace}`. `trace` is not a declared output of `cook`, and the response has no OGC results-document structure. A strict client may reject it.
- **Records conformance URIs.** The declared `…/conf/core` and `…/conf/json` may not match the published Records 1.0 class names, and records declare `conformsTo: …/ogcapi-records-1/1.0/req/record-core`, a requirement URI, not a conformance-class URI. Check both against the standard, and run OGC's Records test suite.
- **Items parameters.** `items` lack `bbox`, `datetime`, `type` and `ids`.

Second plugfest. Varidian put a recipe around their own *Subsidence Analysis* process (`registry.playground.planspace.nl`) into the `playground-3` slot and ran it through our Wizard and Cook; two results are saved in Hub Datasets (section 3). It showed two things we then built:

- A user facing someone else's process does not know what to enter. The process description already says it, in standard fields (titles, descriptions, defaults, limits), so the Wizard now reads it (section 5.4) and links to it.
- Names drifted between screens: the step's key on one, the Operation's label nowhere, and the process description's title on another. One naming rule now applies everywhere (section 5.1).

11. Findings

Twenty-five findings, grouped by the standard or theme they concern.

Recipe format (CWL)

1. **CWL works as a recipe format, but not unextended.** It cannot say which OGC API Process a step is (`nldt:processLink`) or which alternatives a hub may choose (`nldt:compatibleTools`).
2. **CWL's `Operation` class fits remote processes.** An abstract step maps one-to-one onto an OGC API Process, with no `CommandLineTool` involved. CWL's standard requirements (for example `DockerRequirement` and `ResourceRequirement`) describe running a tool locally; none applies to an `Operation` that calls a remote process.
3. **Data inputs need no extension.** `File` + `format` separates ingredients from parameters and maps cleanly onto OGC `href` inputs.
4. **CWL and OGC API Processes use different type systems.** CWL is enough for a recipe's structure, not for describing process inputs as precisely as JSON Schema. A CWL-based format needs a bridge (`nldt:jsonSchema`), ideally a reference rather than a copy.
5. **CWL expressions are JavaScript.** A lightweight cook either embeds a JavaScript engine or supports a pattern-matched subset of `valueFrom`, as ours does.

6. **CWL's extension rule is enforced by tooling.** Extension fields need a namespace prefix declared in `$namespaces`. Our first, OpenAPI-style `x-` names made every recipe invalid for `cwltool --validate`; with a declared `nldt:` namespace, all pass.
7. **A purpose-built cook checks less than a CWL validator.** `cwltool` caught a real type error our Registry had accepted (an enum list wired into a `string[]` input in the roofer recipe). Our reader validates structure and wiring, not types; running the reference validator alongside is cheap.
8. **CWL has no field for media types of non-File values.** `format` is only defined for Files, so requesting the media type of a non-File process input or output has no standard CWL field; our use of `format` there only means something to our Cook.
9. **One reader of the recipe avoids drift.** Only the Registry parses CWL; everything else asks it for the DAG and schema (section 4.2). The tool catalogue was not centralised, and now exists in four copies (section 5.2).
10. **A step and its app need separate names.** CWL already gives a step two `labels`: the step's own, for what it does in the recipe, and its Operation's, for the app that does it. Showing them consistently (section 5.1) matters once apps can be swapped: the step stays, the app changes. Without one rule, a user sees three different names for the same thing.

OGC API Processes

11. **Process descriptions don't say which inputs are datasets.** Roofer describes its point-cloud input only in prose, so a platform cannot infer data ingredients; the recipe has to declare them.
12. **Async execution is mandatory in practice.** Long processes outlive HTTP and proxy timeouts, so the Cook always requests async and polls.
13. **Media types are a gap for 3D and point-cloud data.** LAZ has no IANA-registered media type; `application/vnd.laszip` is a convention.
14. **Version.** We implement Part 1: Core 1.0 and chain through CWL, not Part 3 (Workflows) nested execution.
15. **A recipe can itself be a process.** Wrapping the whole Cook as one OGC API Process (`cook`) lets any Processes client run any recipe without knowing CWL.
16. **By-reference data must be reachable from the process, not the browser.** An address that only the user's own machine or a private network can reach is useless to an external process. The Cook rewrites URLs only for its own tools.
17. **Process descriptions already carry what a parameter form needs.** Titles, descriptions, defaults and limits are standard fields of an OGC API Process description and its JSON Schema. A client can read them at display time and fill a recipe's parameter form with them, without copying them into the recipe (section 5.4).
18. **They rarely carry an example for structured inputs.** A default works for a number; a GeoJSON area has no sensible default. JSON Schema's `examples` keyword exists for this, but the processes we used do not provide it, so a user still has to know the shape of the value.

Catalogue and metadata

19. **Catalogue and recipe metadata are not yet aligned.** OGC API Records carries discovery metadata well; DCAT-AP-NL has no recipe concept.
20. **There is no standard link relation for "this record's executable recipe".** A relation could be proposed to OGC.

App Store operation

21. **Editing is shared and unversioned.** Saving a recipe overwrites it for everyone. There is no versioning, ownership or authentication: fine for a sandbox, not for an App Store.
22. **Swapping tools needs shared knowledge of each alternative's inputs and outputs.** Ours is a built-in catalogue, so swaps are limited to our own four tools. Inputs are matched by position, not by meaning.

Access, delegation and cost

These three were outside the scope of this research, but became clear while building it, and they decide whether a recipe App Store can work in practice.

23. **Apps behind a login need delegated access, and we did not research it.** Every app in this testbed is open: anyone may call it, without an account, key or token. Real apps, commercial or governmental, usually sit behind a secure gateway and need credentials. In a recipe, it is not the user who calls an app but the Cook, on the user's behalf. So the Cook would need to receive, hold and pass on access for each app in the recipe: a token for the user, valid for that app, for the duration of the run (which may be long, section 7.2), without the Cook ever seeing the user's own credentials. General-purpose mechanisms for delegation exist (for example OAuth 2.0 Token Exchange, RFC 8693), but we know of no agreed way to apply them across a chain of OGC API Processes run by a third-party cook. OGC API Processes leaves authentication to each deployment.
24. **Cooks are only interchangeable once access can be delegated.** The recipe architecture separates the recipe (the cookbook) from whoever runs it (the cook), so that in principle any cook can run any recipe, as our `cook` process allows. With authenticated apps, choosing a cook means handing it access to every app in the recipe, and a cook that calls another cook (as Varidian's process is, section 3) would have to pass that access on again. Until there is a trusted, standard way to delegate access to a cook, a real deployment would effectively be tied to one cook it trusts. We see this as the main blocker between this prototype and real-life use.
25. **Commercial apps need a price, and a way to pay.** An App Store implies that some apps cost money. A recipe combines apps from several providers, so a run may have several prices: per run, per amount of data, or by subscription. Who pays whom, when the price is known (before the run, or only after), and how a cook that calls a paid app on the user's behalf is billed, are all open. The Wizard's

setup summary, a basket with an execution fee per app (section 9.3), only gives a feeling of this; the fees are illustrative and nothing is charged.

Questions raised in the stand-ups

- **Recipe vs workflow** (2 Oct). In our build, a recipe is a workflow (the CWL) plus its catalogue record plus its bindings to a hub's data and tools.
- **Input semantics for chaining** (16 Oct). See finding 11 (process descriptions don't mark datasets) and finding 22 (inputs matched by position).
- **A recipe being itself a process** (16 Oct). See finding 15.

12. Recommendations and future work

The recommendations below are addressed to the parties who can act on them.

To Geonovum (nLDT):

- Define recipe metadata in an extended DCAT-AP-NL profile, building on this testbed's proposal (section 8.3).
- Publish the recipe profile and the recipe record as OGC Building Blocks, so other cookbooks can validate against them.
- Agree a small shared vocabulary for recipe extensions (process link, compatible tools, input schema) so cooks and cookbooks from different vendors can exchange recipes.
- Agree media types for 3D and point-cloud formats, starting with LAZ.
- Research delegated access for recipes: how a hub lets a cook use authenticated apps on its behalf, for one run, without handing over its own credentials (findings 23, 24). This is the main step between a sandbox and real use, and is best done together with OGC.
- Explore how pricing and payment would work for recipes that combine paid apps from several providers (finding 25), and how prices appear in recipe metadata (section 8.3).

To OGC:

- Let process descriptions mark an input as a dataset machine-readably, not only in prose.
- Register a link relation for "executable recipe of this record".
- Encourage process descriptions to give structured inputs an `examples` entry, so clients can show users a valid value.
- Describe how an OGC API Processes server accepts delegated access from an orchestrator calling it on a user's behalf, for example as a profile building on OAuth 2.0 Token Exchange, so chained execution can work with authenticated processes.

For our own stack:

- Run `cwltool --validate` as part of the Registry's validate step, to catch type errors.
- Publish a document at the `nldt` namespace IRI defining the three terms (it returns 404 today).
- Reference process schemas instead of copying them into `nldt:jsonSchema`.
- Let `nldt:compatibleTools` reference external processes, resolving their inputs and outputs from the process description instead of a built-in catalogue, and match inputs by meaning rather than position.
- Drop `internalHref` from the recipe format: keep only the public `href`, and let the Cook map known public addresses to internal ones in its own configuration, as it already does for data (section 5.2).
- Let the Rotterdam recipe take a municipality name and fetch its outline from PDOK.
- Let a recipe declare its preferred execution mode and have the client honour it.
- Add versioning, ownership and authentication, plus SSRF protection on process URLs and on the description-fetch endpoint.
- Persist the Cook's job store.
- Retry publishing to the catalogue when CKAN is briefly unavailable, and give the Records Wrapper more than 10 seconds for writes to CKAN: after the redeployment, some catalogue writes failed while CKAN was slow, although the recipes themselves were stored.
- Host a small `echo` test process next to the Cook, so OGC's test suite can check its execution and job handling end to end (section 10).
- Serve an OpenAPI 3.0 document that declares every answer the services give (response schemas, `201`, the OGC `404`s), and close the gaps the Geonovum checker found: `limit` on the process list, the job-list filters on the Cook, job endpoints on the tools, and the Records media types on the catalogue (section 10).
- Express recipe metadata in DCAT-AP-NL in the catalogue.
- Give every step in the bundled and playground recipes a short `label`, and short Operation labels (the `playground-2` ones are copied sentences such as "Create a welcome message string."), since these are the names users see (section 5.1).
- Show a JSON Schema `examples` value as the placeholder of a structured field, once processes provide one.

13. Collaboration

The tender (§7.3) asks participants to document what they expect from each other and how they worked together. We collaborated through bi-weekly stand-ups, plugfests and GitHub Discussions, and by calling each other's live processes.

What we provided to others:

- A public, live stack whose `cook` any OGC API Processes client can call.
- Shared playground recipe slots so participants could test their own processes without our help.

What we used from others:

- Roofer staging processes for a realistic, long-running, structured-input pipeline.
- Geolabs ZOO-Project processes for by-reference data from an independent server.
- Varidian's *Subsidence Analysis* process (`registry.playground.planspace.nl`), itself a cook, as a step in our `playground-3` recipe. Its well-described inputs (titles, descriptions, defaults, limits) are what the Wizard's parameter form now reads.
- PDOK WFS (Kadaster) for live boundary data.

What we expect from process providers:

- Publicly reachable, unauthenticated OGC API Processes for testing.
- Asynchronous execution for anything long-running.
- Precise JSON Schemas for inputs, with datasets marked as such and media types declared.
- Stable process descriptions, or a version signal when they change.
- For every input: a `title`, a `description`, a `default` where one makes sense, and limits (`minimum`, `maximum`) in its JSON Schema. A client can turn these into a usable form without copying anything into the recipe (finding 17).
- For structured inputs such as GeoJSON, a JSON Schema `examples` entry with one small, valid value (finding 18).

14. Availability, licensing and running the stack

The client and backend stay online for at least six months after the testbed ends, as the tender requires.

Item	Location
Live client (App Store UI)	fi-group.github.io/nldt-tb2
Live backend	nldt-tb2.clearly.app, services under `/api/*`

Item	Location
Testing without the UI	The same APIs, directly; endpoints and examples in Appendix B
Source code	https://github.com/Geonovum/DTaaS-Testbed2026-Phase2/tree/main/Topic4-FutureInsight
Demo video	https://github.com/Geonovum/DTaaS-Testbed2026-Phase2/blob/main/Topic4-FutureInsight/future_insight_demo.mp4, in the same folder

Licensing. The code is published under EUPL-1.2, the licence of the Geonovum testbed repository; the report and other testbed outcomes stay under CC BY 4.0. Vendored Leaflet keeps its own licence.

Running your own copy. The live stack above is the reference installation. To run another copy, for example for development, on a machine with Docker:

```shell
docker compose up --build
python services/records-wrapper/scripts/bootstrap_ckan.py
# once: CKAN organisation + API token
python services/cookbook-registry/scripts/seed_demo_recipe.py
# store every bundled recipe
```

The UI is then served by the `ui` container, and every service on its own port, as listed in `docker-compose.yml`. The seed script needs PyYAML installed on the host. On first start, CKAN and the services need a few minutes before `bootstrap_ckan.py` works. Run the seed only once CKAN is up: a recipe is always stored, but if CKAN is not ready, publishing it to the catalogue fails and the script reports "STORED BUT NOT PUBLISHED". Running the script again is safe. Seeding overwrites all stored recipes, including whatever participants saved in the playground slots. After the `nldt:` namespace change, re-seeding is required, because old stored recipes no longer parse.

Behind TLS or Cloudflare, add the gateway overlay: `docker compose -f docker-compose.yml -f docker-compose.gateway.yml up -d --build`.

Per-deployment settings (`PUBLIC_HOST`, `PUBLIC_TOOL_BASE`, `GATEWAY_PORT`) go in `.env`.

Limitations. All services are open, with no authentication and open CORS. Processes must be unauthenticated OGC API Processes servers. There are two unprotected server-side fetches of user-supplied URLs (process URLs in recipes, and `GET /processes/fetch-description`), so the stack is unsuitable for untrusted recipe authors.

Appendix A: Design decisions and turning points

The decisions that shaped the solution, oldest first. Most are a turning point: we started one way, ran into a limitation, and changed course. The full, dated log up to 22 October is `DECISIONS.md` in the repository; later entries come from our working notes.

Date (2026)	Decision	What came before, and what we learned
8 Sep	Build the infrastructure as the real system; only the apps a recipe calls may stay trivial. Start with `(a + b) × c`	The infrastructure is the deliverable; the apps are interchangeable by design. A trivial recipe proves the whole chain (discovery, parameters, two steps, piping, result) before any domain complexity
8 Sep	CWL as the recipe format, rather than Arazzo or our own JSON format	CWL is a mature, externally governed standard for chaining discrete steps. Our own format would have needed its own tooling and governance (section 5)
8 Sep	Each step is an inline CWL `Operation`, one per OGC API Process; recipes are read by a small purpose-built parser, not a full CWL engine	`Operation` is CWL's own "abstract step that calls something elsewhere". Later we learned the parser is too lenient: it missed a type error that `cwltool` caught, and silently ignores `scatter` and `when` (finding 7)
8 Sep	OGC API Processes Part 1: Core 1.0, with chaining	We first chose v2.0 with Part 3 (Workflows) for

Date (2026)	Decision	What came before, and what we learned
	done by CWL inside the Cook	chaining. Part 3 was a draft whose conformance classes we could not verify; Part 1 is stable, and CWL already describes the chain
8 Sep	The Cook is itself an OGC API Process (`cook`), reading the recipe's plan fresh on every run	Any OGC API Processes client can run any recipe without knowing CWL, and a saved recipe needs no redeployment. The cost: a recipe never appears as its own process with typed inputs; clients only see `cook` (finding 15)
8 Sep	Recipes carry their own process link (`nldt:processLink`); a static table of our tools is only a fallback	First, the Cook looked every process up in a static table: every new app meant a code change. With the link in the recipe, we swapped multiply for a divide app through the UI without touching code: `(2 + 3) / 4 = 1.25`
9 Sep	Shared "playground" recipe slots for the plugfests, instead of a recipe-creation UI	Every mechanism needed (edit, validate, save, run) already existed; the gap was only "not enough recipes to hand out". The price: anyone can overwrite a slot, and there is no version history (finding 21, section 3)

Date (2026)	Decision	What came before, and what we learned
10 Sep	Catalogue backend: real CKAN 2.11 behind the Records Wrapper, instead of our own SQLite store	The tender's architecture names CKAN. The switch touched only the wrapper's storage module, which proved the seam. CKAN's 2.9 and 2.10 images would not start; 2.11 did
Sep–Oct	The Cook always asks each process for asynchronous execution and follows its job; independent steps run in parallel	The first roofer recipe (28 Sep) failed on a 10-second synchronous timeout, and on inputs that were not plain numbers or text. Real processes outlive any HTTP timeout, so async became the default for every step (finding 12). The same attempt showed that the Cook could not send credentials: the first sign of the access problem in findings 23–24
16 Oct	In the recipe editor, the CWL text is the single source of truth: text and diagram side by side, kept in sync, with the guided builder as an optional help, and nothing wired automatically	We first made the guided builder the default. But people arrive with CWL to paste, and automatic wiring connects things behind their backs. The builder itself answered the first plugfest's lesson: writing a recipe required knowing how we had structured the CWL

Date (2026)	Decision	What came before, and what we learned
Oct	One public address for everything (UI and `/api/*` behind one gateway), and the UI also on GitHub Pages	Cloudflare's proxy only forwards a fixed list of ports, so per-service ports could not work behind TLS. The UI has no server logic, so it can run anywhere. The side effect, found only on 28 October with OGC's test suite: services built links without their `/api/…` prefix, until each was told its public prefix (section 10)
22 Oct	The repository was narrowed to this testbed only, removing earlier mock-only flows	Flows such as vendor binding, semantic alignment and partner pricing only ever existed as mock-ups. Removing them makes clear what the testbed actually demonstrates
22 Oct	Data ingredients are declared with standard CWL `File` + `format` and passed by reference: the process fetches the data itself	Before, the UI knew a recipe's data inputs from a hard-coded list per recipe, so a recipe saved in a playground slot could never have one; the roofer recipe looked as if it ran without data. We rejected a new extension field (CWL already has `File`) and reading it from process descriptions (roofer states it only in prose, finding 11)

Date (2026)	Decision	What came before, and what we learned
22 Oct	Process-specific input shapes are handled in the recipe, with standard CWL `valueFrom`; the Cook recognises a fixed set of patterns and never runs JavaScript	Even with `File`, roofer wanted `[{"kind": "url", "url": …}]` rather than an OGC `href`. CWL expressions are JavaScript; a small cook either embeds a JavaScript engine or, like ours, supports a subset (finding 5)
22 Oct	`nldt:jsonSchema`: an optional bridge for parameters CWL's types cannot describe	CWL cannot express `oneOf`, `const` or `pattern`, which roofer's inputs use. Its weakness, a pasted copy that goes stale, led to the 28 October change below
22 Oct	Extension fields renamed from `x-…` to a declared `nldt:` namespace; the old names are rejected outright	`cwltool --validate` rejected every recipe: CWL only allows extension fields with a declared prefix. Silently ignoring the old names would have dropped a step's process link and failed only at run time. Lesson: deploying a format change means re-storing every recipe (finding 6)

Date (2026)	Decision	What came before, and what we learned
27 Oct	A failed run says whose failure it is: the Cook's, the Registry's or a step's app	A recipe calls processes the hub does not control; users could not tell a platform fault from a third-party one (section 7.4)
27 Oct	Job addresses returned by a remote process are resolved relative to that process's own address	Some servers answer with a relative `/jobs/{jobID}`, which breaks polling. Chaining processes from different vendors surfaces small differences like this
28 Oct	One naming rule: an app is named by its CWL Operation `label`, a step by its step `label`	Each screen showed whatever name was nearest: the step's key, or a title from the remote process description; the recipe's own app name never reached the UI. Once apps can be swapped, the step and its app must have separate names (section 5.1, finding 10)
28 Oct	The Wizard completes the parameter form from each app's own OGC API Process description, at display time; the recipe always wins	Copying schemas into recipes goes stale. Process descriptions already carry titles, descriptions, defaults and limits in standard fields (section 5.4, finding 17)
28 Oct	Our OGC API Processes services tested with OGC's official test suite, and fixed:	Until then we had only checked them by hand and with our own clients, which

Date (2026)	Decision	What came before, and what we learned
	OGC's error format, links to the API definition, complete process and job lists, `result-not-ready`	were lenient. The suite found gaps every service shared; it also showed that most of its execute tests need OGC's `echo` test process, which a service with specific inputs cannot replace (section 10)
28 Oct (realisation)	`internalHref` is a deployment detail that leaked into the recipe format; we recommend dropping it (not yet done)	It works, but a recipe should name *which* process to call, not how one installation reaches it (section 5.2)
28 Oct (realisation)	Authentication, delegated access and pricing are the main gaps between this sandbox and real use	Out of scope by design, but every step of the build assumed open, free apps. A real cook would need delegated access to each app, and cooks are only interchangeable once that can be delegated (findings 23–25)

Appendix B: Using the stack without the UI (API reference)

Everything the App Store UI does goes through public APIs, so the whole solution can also be tested with any HTTP client (curl, Postman, a script). All services are under one address, with no login:

```
None
https://nldt-tb2.clearly.app/api
```

All requests and answers are JSON, except the raw CWL (`application/cwl+yaml`). Every example below was run against the live stack while writing this report.

B.1 Endpoints

#	What	Request	Notes
1	Find recipes in the catalogue	`GET /records/collections/recipes/items`	OGC API Records; optional `q`, `limit`, `offset`. Each record links to its CWL
2	One catalogue record	`GET /records/collections/recipes/items/{recipeId}`	
3	List recipes in the cookbook	`GET /cookbook/recipes`	Ids, titles, descriptions
4	A recipe's CWL	`GET /cookbook/recipes/{recipeId}`	The stored text, verbatim

#	What	Request	Notes
5	What parameters it takes	`GET /cookbook/recipes/{recipeId}/schema`	JSON Schema of the recipe's inputs
6	Its execution plan	`GET /cookbook/recipes/{recipeId}/dag`	Section 6.1
7	Its plan with an app swapped	`POST /cookbook/recipes/{recipeId}/effective`	Body `{"toolOverrides": {"<stepId>": "<toolId>"}}`; returns `{schema, dag}`; nothing is stored
8	Check a CWL draft	`POST /cookbook/recipes/validate`	Body `{"cwl": "<CWL text>"}`; always 200, with `valid` true or false
9	Describe one of our apps	`GET /tools/{tool}/processes/{processId}`	`{tool}` is `add`, `multiply`, `multiply3` or `geo-buffer` (table B.2)

#	What	Request	Notes
10	Call one of our apps directly	`POST /tools/{tool}/processes/{processId}/execution`	Body `{"inputs": {…}}`; answers at once (synchronous)
11	Describe the Cook	`GET /orchestrator/processes/cook`	Its inputs: `recipeId`, `parameters`, `toolOverrides`
12	Run a recipe and wait for the result	`POST /orchestrator/processes/cook/execution`	Body `{"inputs": {"recipeId": …, "parameters": {…}, "toolOverrides": {…}}}`; answers `{result, trace}` when the run is done
13	Start a recipe run as a job	Same as 12, with header `Prefer: respond-async`	Answers `201 Created` at once, with the job's first status
14	Follow a job	`GET /orchestrator/jobs/{jobID}`	Status, `progress` (0–100), message, and every step's status

#	What	Request	Notes
15	Get a job's result	`GET /orchestrator/jobs/{jobID}/results`	`{result, trace}`, once the job is `successful`
16	List all jobs	`GET /orchestrator/jobs`	Every job since the Cook last started (jobs are kept in memory only)
17	Cancel a job	`DELETE /orchestrator/jobs/{jobID}`	Stops the run and dismisses any remote job
18	List datasets (ingredients and saved results)	`GET /hub/datasets`, then `GET /hub/datasets/{datasetId}/resources`	
19	A dataset's data, as a process would fetch it	`GET /hub/datasets/{datasetId}/resources/{resourceId}/data`	Use this address as a data input (examples B.3)

Links in answers. The addresses the services return (a job's `Location` header, `self`, `monitor`, `results` and `execute` links) can be followed directly. Until the October 2026 redeployment, they left out the `/api/<service>` part of the address (section 10); with an older deployment, build the address yourself: `/api/orchestrator/jobs/{jobID}`.

B.2 Our own apps

These four apps are ours, and can be called directly. The playground recipes call other parties' processes (roofer, Geolabs ZOO-Project, Varidian); run those through the Cook, as their owners intended, rather than calling them directly.

App	Request (under `/api/tools`)	Inputs	Output
Add two numbers	`/add/processes/add-two-numbers/execution`	`a`, `b` (numbers)	`sum`
Multiply two numbers	`/multiply/processes/multiply-two-numbers/execution`	`x`, `y` (numbers)	`product`
Multiply three numbers	`/multiply3/processes/multiply-three-numbers/execution`	`x`, `y`, `z` (numbers)	`product`
Buffer a geometry	`/geo-buffer/processes/buffer-geometry/execution`	`geometry` (GeoJSON, given directly or by reference), `distance` (metres)	`bufferedGeometry` (GeoJSON Feature)

B.3 Examples

In the examples, `$B` stands for `https://nldt-tb2.clearly.app/api`.

Call app 1 (add) directly:

```shell
curl -X POST "$B/tools/add/processes/add-two-numbers/execution" \
  -H "Content-Type: application/json" \
  -d '{"inputs": {"a": 2, "b": 3}}'
# → {"outputs": {"sum": 5.0}}
```

Call app 2 (multiply) directly:

```shell
curl -X POST "$B/tools/multiply/processes/multiply-two-numbers/execution" \
  -H "Content-Type: application/json" \
  -d '{"inputs": {"x": 5, "y": 4}}'
# → {"outputs": {"product": 20.0}}
```

Buffer a geometry, given directly (a point in Rotterdam, by 100 m):

```shell
curl -X POST "$B/tools/geo-buffer/processes/buffer-geometry/execution" \
  -H "Content-Type: application/json" \
    -d '{"inputs": {"geometry": {"type": "Point", "coordinates": [4.4777, 51.9244]}, "distance": 100}}'
# → {"outputs": {"bufferedGeometry": {"type": "Feature", "geometry": {"type": "Polygon", …}}}}
```

Buffer a geometry, given by reference (Rotterdam's outline, fetched live from PDOK through Hub Datasets, by 500 m). The app downloads the data itself:

```shell
curl -X POST "$B/tools/geo-buffer/processes/buffer-geometry/execution" \
  -H "Content-Type: application/json" \
  -d '{"inputs": {
                                                "geometry": {"href": "'"$B"'/hub/datasets/ds-pdok-rotterdam-boundary/resources/res-pdok-rotterdam-outline/data",
                      "type": "application/geo+json"},
        "distance": 500}}'
```

Run a recipe and wait for the result (`recipe-add-multiply-demo`, (2 + 3) × 4):

```shell
curl -X POST "$B/orchestrator/processes/cook/execution" \
```

```
  -H "Content-Type: application/json" \
   -d '{"inputs": {"recipeId": "recipe-add-multiply-demo", "parameters": {"a":
2, "b": 3, "c": 4}}}'
# → {"result": {"result": 20.0},
#    "trace": [{"step": "add", "process_id": "add-two-numbers", "inputs": {"a":
2, "b": 3}, "outputs": {"sum": 5.0}, …},
#                  {"step": "multiply", "process_id": "multiply-two-numbers",
"inputs": {"x": 5.0, "y": 4}, "outputs": {"product": 20.0}, …}]}
```

The trace shows the piping: `add`'s `sum` (5) became `multiply`'s `x`.

Run the same recipe with an app swapped (multiply → multiply three numbers, which needs a third value, `z`). The recipe itself is not changed:

Shell

```
curl -X POST "$B/orchestrator/processes/cook/execution" \
  -H "Content-Type: application/json" \
  -d '{"inputs": {"recipeId": "recipe-add-multiply-demo",
                  "parameters": {"a": 2, "b": 3, "c": 4, "z": 10},
                  "toolOverrides": {"multiply": "multiply-three-numbers"}}}'
# → {"result": {"result": 200.0}, …}
```

To see which parameters a swap adds before running it, ask for the effective plan (endpoint 7) with the same `toolOverrides`.

Run a recipe as a job, with a data input (`recipe-rotterdam-buffer-demo`, buffering Rotterdam's outline by 250 m). A data input is a CWL File: a link to the data, never the data itself. This is how the App Store UI runs every recipe:

Shell

```
curl -i -X POST "$B/orchestrator/processes/cook/execution" \
  -H "Content-Type: application/json" -H "Prefer: respond-async" \
  -d '{"inputs": {"recipeId": "recipe-rotterdam-buffer-demo", "parameters": {
        "geometry": {"class": "File",
                                                                "location":
"'"$B"'/hub/datasets/ds-pdok-rotterdam-boundary/resources/res-pdok-rotterdam-ou
tline/data",
                                                                  "format":
"https://www.iana.org/assignments/media-types/application/geo+json"},
        "bufferDistance": 250}}}'
# → HTTP/1.1 201 Created
#   Location: https://nldt-tb2.clearly.app/api/orchestrator/jobs/85e1ea92-…
#      {"jobID": "85e1ea92-…", "processID": "cook", "status": "accepted",
"recipeId": "recipe-rotterdam-buffer-demo", …}
```

```
curl "$B/orchestrator/jobs/85e1ea92-…"            # repeat every few seconds
# → {"status": "running", …}  then
#   {"status": "successful", "progress": 100, "message": "1 of 1 steps done",
#      "steps": [{"id": "buffer", "label": "Step 1 — Buffer", "processId":
"buffer-geometry", "status": "successful", …}], …}

curl "$B/orchestrator/jobs/85e1ea92-…/results"
# → {"result": {"bufferedOutline": {"type": "Feature", "geometry": {"type":
"Polygon", …}}}, "trace": […]}
```

List all jobs of the Cook:

Shell

```
curl "$B/orchestrator/jobs"
#   →   {"jobs":   [{"jobID":   "…",   "status":   "successful",   "recipeId":
"playground-1", …}, …]}
```

Runs started through the API are not saved to Hub Datasets; saving the result is something the App Store UI does after a run (section 7.1).

Appendix C: The playground recipes, annotated

The three playground slots can be overwritten by anyone (section 3), so this appendix keeps them as they were at the time of writing. These are the bundled versions, the ones Reset restores, from `services/cookbook-registry/recipes/` in the repository. The files' own comments are replaced by short notes marked `# ←`; nothing else differs. The CWL constructs themselves are explained in section 5.

C.1 `playground-1`: roofer building reconstruction

Three of the roofer's processes in a chain: validate point clouds → reconstruct buildings → convert the model to CityJSON and/or GeoPackage. Each step's output feeds the next step, reshaped with `valueFrom`.

```
None
cwlVersion: v1.2
class: Workflow
id: playground-1
label: "Playground Recipe 1"
doc: >
  A shared slot for trying out your own recipe. Anyone can edit and save it,
  so it holds whatever was saved last — the starter roofer pipeline (validate,
  reconstruct and convert buildings), or someone else's work in progress. To
  start fresh, open Edit Recipe → Orchestrator and click Reset: that loads the
  starter version, whose comments explain where to begin. Then Validate and
  Save.

requirements:                              # ← for the CWL standard only: our Cook
ignores these two lines (section 5.1)
  StepInputExpressionRequirement: {} # ← declares that steps use "valueFrom" to
reshape inputs (all three steps do)
   InlineJavascriptRequirement: {}       # ← declares that those expressions
contain JavaScript (the two "map" lines)

$namespaces:
   iana: https://www.iana.org/assignments/media-types/     # ← standard media
types, to name file formats
  nldt: https://fi-group.github.io/nldt-tb2/ns#         # ← our extensions (all
fields starting with nldt:)

inputs:
   point_clouds:                               # ← a DATA input: picked from Hub
Datasets, not typed in
     type: File[]                      # ← one or more files, passed by reference
(a URL)
     format: iana:application/vnd.laszip    # ← LAZ point clouds (common, but not
IANA-registered)
```

```
    label: "Point clouds"
      doc: "The LAZ point cloud files to validate and reconstruct buildings
from."
  bag:            # ← a PARAMETER: which BAG buildings to reconstruct
    type: Any
    label: "BAG selection"
    doc: >
      Which BAG buildings to reconstruct: a whole BAG asset, a list of
      building ids, or every building within an area (WKT in EPSG:28992).
    default:                          # ← pre-filled value in the Wizard
      kind: buildings
      building_ids: ["0000000000000001", "0000000000000002"]
    nldt:jsonSchema:                    # ← the exact allowed shape, copied from
roofer's process description (section 5.4)
      oneOf:                            # ← CWL types can't say "one of these
three shapes"; the Wizard reads this instead
        - title: "By BAG asset"       # ← option 1: a whole BAG asset
          type: object
          additionalProperties: false
          required: [kind, asset_id]
          properties:
            kind: { const: asset, type: string }
            asset_id: { type: integer, exclusiveMinimum: 0 }
        - title: "By building ids"    # ← option 2: a list of building ids
          type: object
          additionalProperties: false
          required: [kind, building_ids]
          properties:
            kind: { const: buildings, type: string }
            building_ids:
              type: array
              minItems: 1
              uniqueItems: true
              items: { type: string, pattern: "\\S" }
        - title: "By area"              # ← option 3: every building inside a
polygon
          type: object
          additionalProperties: false
          required: [kind, wkt, crs]
          properties:
            kind: { const: area, type: string }
            wkt: { type: string, pattern: "\\S" }
            crs: { const: "EPSG:28992", type: string }
  name:
    type: string?                     # ← "?" means optional
    label: "Model name"
    doc: "Optional name for the reconstructed model."
  config:
```

```
    type: Any?
    label: "Reconstruction config"
    doc: "Optional roofer configuration overrides, as a JSON object."
    nldt:jsonSchema:
      type: object
      additionalProperties: true
  formats:
    type:
      type: array
      items:
        type: enum           # ← a fixed list of choices: gpkg and/or cityjson
        symbols: [gpkg, cityjson]
    label: "Output formats"
    doc: "Which file formats to convert the model into."
    default: [cityjson]

outputs:                      # ← the recipe returns the result of every step
  validation_report:
    type: Any
    outputSource: validate/validation_report
  building_model:
    type: Any
    outputSource: reconstruct/building_model
  converted_model:
    type: Any
     outputSource: convert/converted_model     # ← the final CityJSON/GeoPackage
export

steps:
  validate:          # ← STEP 1: register the point clouds with roofer
    label: "Validate point clouds"
    run:
       class: Operation      # ← CWL's abstract step: "call something that runs
elsewhere"
      id: roofer:validate_point_cloud:v1 # ← the OGC API Process id
      label: "Validate Point Cloud"
      doc: "Calls the 'roofer:validate_point_cloud:v1' OGC API Process."
      nldt:processLink:   # ← where this process lives: roofer's staging server
        rel: process
        type: application/json
                                                                          href:
"https://processes.staging.roofer-online.nl/ogcapi/processes/roofer:validate_po
int_cloud:v1"
                                                                 internalHref:
"https://processes.staging.roofer-online.nl/ogcapi/processes/roofer:validate_po
int_cloud:v1"
      inputs:
        point_clouds: { type: Any }
```

```
      outputs:
        validation_report: { type: Any }
    in:
      point_clouds:
        source: point_clouds          # ← takes the recipe's point clouds…
            valueFrom: '$(self.map(function(f){ return {"kind": "url", "url":
f.location}; }))'
                                      # ← …and reshapes each file into roofer's
own form, {"kind": "url", "url": …},
                                                #   since roofer does not take OGC
by-reference inputs ({"href": …})
    out: [validation_report]          # ← contains an id for each point cloud

  reconstruct:                        # ← STEP 2: build the 3D model
    label: "Reconstruct buildings"
    run:
      class: Operation
      id: roofer:reconstruct_buildings:v1
      label: "Reconstruct Buildings"
      doc: "Calls the 'roofer:reconstruct_buildings:v1' OGC API Process."
      nldt:processLink:
        rel: process
        type: application/json
                                                                          href:
"https://processes.staging.roofer-online.nl/ogcapi/processes/roofer:reconstruct
_buildings:v1"
                                                                  internalHref:
"https://processes.staging.roofer-online.nl/ogcapi/processes/roofer:reconstruct
_buildings:v1"
      inputs:
        point_cloud_ids: { type: "int[]" }
        bag: { type: Any }
        name: { type: "string?" }
        config: { type: "Any?" }
      outputs:
        building_model: { type: Any }
    in:
      point_cloud_ids:
         source: validate/validation_report    # ← CHAINING: the output of step
1…
                    valueFrom:   "$(self.point_clouds.map(function(p){   return
p.point_cloud_id; }))"
                                        # ← …from which the point cloud ids are
picked out
       bag: bag                         # ← the other inputs come straight from
the Wizard
      name: name
      config: config
```

```
    out: [building_model]            # ← contains the id of the new 3D model

  convert:                           # ← STEP 3: export the model
    label: "Convert format"
    run:
      class: Operation
      id: roofer:convert_format:v1
      label: "Convert Format"
      doc: "Calls the 'roofer:convert_format:v1' OGC API Process."
      nldt:processLink:
        rel: process
        type: application/json
                                                                        href:
"https://processes.staging.roofer-online.nl/ogcapi/processes/roofer:convert_for
mat:v1"
                                                                    internalHref:
"https://processes.staging.roofer-online.nl/ogcapi/processes/roofer:convert_for
mat:v1"
      inputs:
        model_3d_id: { type: int }
        formats:
          type: { type: array, items: { type: enum, symbols: [gpkg, cityjson] }
}   # ← must match the recipe input's type (section 5.1)
      outputs:
        converted_model: { type: Any }
    in:
      model_3d_id:
         source: reconstruct/building_model    # ← CHAINING: the output of step
2…
          valueFrom: "$(self.model_3d_id)"      # ← …from which the model id is
picked out (a field path, no JavaScript)
      formats: formats
     out: [converted_model]              # ← links to the CityJSON/GeoPackage
files; the CityJSON is shown on the preview map
```

C.2 `playground-2`: Geolabs ZOO-Project sampler

Five processes on Geolabs' ZOO-Project server. Four of them do not depend on each other and run in parallel; one chain, Buffer → GetArea, passes Buffer's GML output on by value.

```
None
cwlVersion: v1.2
class: Workflow
id: playground-2
label: "Playground Recipe 2"
```

```
doc: >
  A shared slot for trying out your own recipe. Anyone can edit and save it,
  so it holds whatever was saved last — the starter Geolabs process sampler
  (Hello World services and GDAL/OGR geometry operations on Geonovum's
  ZOO-Project testbed server), or someone else's work in progress. To start
  fresh, open Edit Recipe → Orchestrator and click Reset: that loads the
  starter version, whose comments explain where to begin. Then Validate and
  Save.

$namespaces:
  iana: https://www.iana.org/assignments/media-types/
  nldt: https://fi-group.github.io/nldt-tb2/ns#

inputs:
   name:                                  # ← a PARAMETER: who the Hello World
services greet
    type: string
    label: "Name"
    doc: "Who the Hello World services greet."
    default: nLDT
   polygon:                              # ← a DATA input, e.g. the hub's "Sample
Polygon" dataset
    type: File
    format: iana:application/geo+json
    label: "Polygon"
    doc: "A GeoJSON Polygon geometry file, used by the geometry steps."
   buffer_distance:                        # ← a PARAMETER, in the polygon's own
coordinate units
    type: float
    label: "Buffer distance"
    doc: "How far to buffer the polygon, in its own coordinate units."
    default: 10

outputs:                                # ← one output per step
  greeting_python:
    type: string
    outputSource: hello_python/Result
  greeting_javascript:
    type: string
    outputSource: hello_javascript/result
  centroid:
    type: Any
    outputSource: centroid/Result
  buffered_polygon:
    type: string
    outputSource: buffer/Result
  buffered_area:
    type: float
```

```
      outputSource: area/Area              # ← for the 100 x 100 sample square
buffered by 10: about 14314

steps:
  hello_python:                          # ← runs in parallel with the next three
steps
    label: "Hello World (Python)"
    run:
      class: Operation
      id: HelloPy
      label: "Create a welcome message string."
      doc: "Calls the 'HelloPy' OGC API Process."
        nldt:processLink:                 # ← where this process lives: Geolabs'
ZOO-Project server
        rel: process
        type: application/json
        href: "https://host1.tb.geonovum.geolabs.fr/ogc-api/processes/HelloPy"
                                                                    internalHref:
"https://host1.tb.geonovum.geolabs.fr/ogc-api/processes/HelloPy"
      inputs:
        a: { type: string }
      outputs:
        Result: { type: string }
    in:
       a: name                           # ← recipe input "name" → process input
"a"
    out: [Result]

  hello_javascript:
    label: "Hello World (JavaScript)"
    run:
      class: Operation
      id: hellojs
      label: "HelloWorld Service in JavaScript"
      doc: "Calls the 'hellojs' OGC API Process."
      nldt:processLink:
        rel: process
        type: application/json
        href: "https://host1.tb.geonovum.geolabs.fr/ogc-api/processes/hellojs"
                                                                    internalHref:
"https://host1.tb.geonovum.geolabs.fr/ogc-api/processes/hellojs"
      inputs:
        S: { type: string }
      outputs:
        result: { type: string }
    in:
      S: name
    out: [result]
```

```
  centroid:
    label: "Centroid"
    run:
      class: Operation
      id: Centroid
      label: "Computes the centroid of a polygon."
      doc: "Calls the 'Centroid' OGC API Process."
      nldt:processLink:
        rel: process
        type: application/json
        href: "https://host1.tb.geonovum.geolabs.fr/ogc-api/processes/Centroid"
                                                                internalHref:
"https://host1.tb.geonovum.geolabs.fr/ogc-api/processes/Centroid"
      inputs:
          InputPolygon: { type: File, format: "iana:application/json" }    # ←
sent by reference: {"href": …, "type": "application/json"}
      outputs:
          Result: { type: Any, format: "iana:application/json" }           # ←
asks the process for this output as JSON (section 5.1)
    in:
      InputPolygon: polygon
    out: [Result]

  buffer:
    label: "Buffer"
    run:
      class: Operation
      id: Buffer
      label: "Create a buffer around a polygon."
      doc: "Calls the 'Buffer' OGC API Process."
      nldt:processLink:
        rel: process
        type: application/json
        href: "https://host1.tb.geonovum.geolabs.fr/ogc-api/processes/Buffer"
                                                                internalHref:
"https://host1.tb.geonovum.geolabs.fr/ogc-api/processes/Buffer"
      inputs:
        InputPolygon: { type: File, format: "iana:application/json" }
        BufferDistance: { type: float }
      outputs:
          Result: { type: string }      # ← no format: the process's default
output, GML
    in:
      InputPolygon: polygon
      BufferDistance: buffer_distance
    out: [Result]
```

```
  area:                                # ← starts as soon as Buffer has finished
    label: "Area of the buffered polygon"
    run:
      class: Operation
      id: GetArea
      label: "Compute geometry area."
      doc: "Calls the 'GetArea' OGC API Process."
      nldt:processLink:
        rel: process
        type: application/json
        href: "https://host1.tb.geonovum.geolabs.fr/ogc-api/processes/GetArea"
                                                                    internalHref:
"https://host1.tb.geonovum.geolabs.fr/ogc-api/processes/GetArea"
      inputs:
         InputPolygon: { type: string, format: "iana:text/xml" }    # ← sent by
value, as {"value": <GML>, "mediaType": "text/xml"}
      outputs:
        Area: { type: float }
    in:
       InputPolygon: buffer/Result     # ← CHAINING: Buffer's GML output, passed
on unchanged
    out: [Area]
```

C.3 `playground-3`: Varidian's Subsidence Analysis

One step around Varidian's own OGC API Process (section 3): where subsidence stays below a threshold, and so where construction is possible. Every input is passed through unchanged, so this recipe needs no `valueFrom` and no `requirements`.

```
None
cwlVersion: v1.2
class: Workflow
id: playground-3
label: "Playground Recipe 3"
doc: >
  A shared slot for trying out your own recipe. Anyone can edit and save it,
  so it holds whatever was saved last — Varidian's Subsidence Analysis
  process (where subsidence stays below a threshold, and so where
  construction is possible), or someone else's work in progress. To start
  fresh, open Edit Recipe → Orchestrator and click Reset: that loads this
  bundled version. Then Validate and Save.
$namespaces:
  nldt: https://fi-group.github.io/nldt-tb2/ns#
  iana: https://www.iana.org/assignments/media-types/
```

```
inputs:                                    # ← all four are PARAMETERS, typed or
pasted into the Wizard
  gw:
    type: float                          # ← groundwater level, in metres relative
to NAP
  thickness:
    type: float                          # ← surcharge (fill) thickness in metres,
for areas that don't set their own
  geojson:
    type: Any                                  # ← a GeoJSON FeatureCollection; a
parameter, not a data input (section 3)
    label: "Surcharge area(s)"
    doc: "Paste a GeoJSON FeatureCollection."
  threshold:
    type: float                             # ← the subsidence threshold to test
against

outputs:
  buildArea:                                # ← the area(s) where construction is
possible
    type: File
    outputSource: varidian-cook_subsidence-analysis/buildArea    # ← taken from
the step below, output "buildArea"

steps:
  varidian-cook_subsidence-analysis: # ← no step label, so the UI shows this
key (section 5.1)
    run:
      class: Operation
       id: varidian-cook_subsidence-analysis    # ← the OGC API Process id at
Varidian
      label: "Subsidence Analysis App"
      doc: "Calls the 'varidian-cook_subsidence-analysis' OGC API Process."
       nldt:processLink:                # ← where this process lives: Varidian's
own server
        rel: process
        type: application/json
                                                                          href:
"https://registry.playground.planspace.nl/processes/varidian-cook_subsidence-an
alysis"
                                                                  internalHref:
"https://registry.playground.planspace.nl/processes/varidian-cook_subsidence-an
alysis"
      inputs:
        gw: { type: float }
        thickness: { type: float }
         geojson: { type: Any }       # ← no format, so the FeatureCollection is
sent as-is,
```

```
                                                  #    not wrapped in {"value": …,
"mediaType": …}
        threshold: { type: float }
      outputs:
        buildArea: { type: File }
     in:                                       # ← wiring: each recipe input → the
process input of the same name
      gw: gw
      thickness: thickness
      geojson: geojson
      threshold: threshold
    out: [buildArea]
```